

INTRODUCTION TO DEEP LEARNING

PART II – ADVANCED CONCEPT AND CNN

Auliya Fitri, Sheetal Sahoo, Sreerag Naveenachandran

**Machine Learning Group
Institute of Data Science**



Schedule

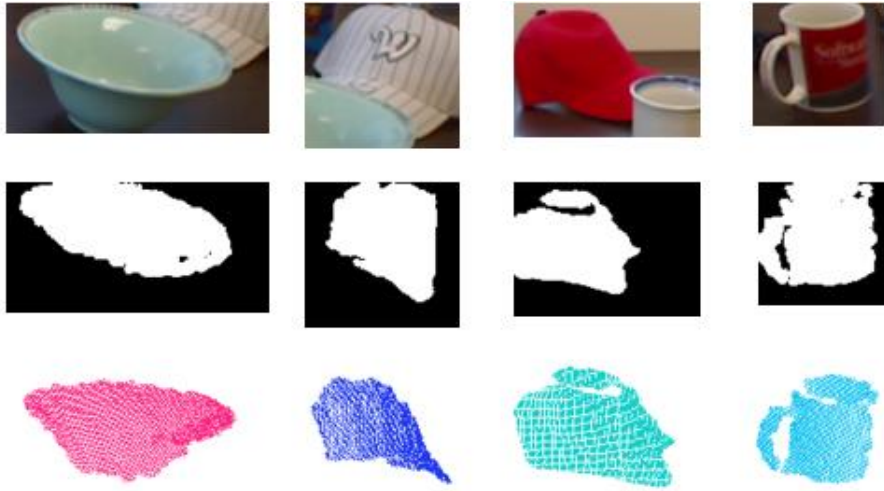


Date	Time	Activity
28.05.2026 Day 1	09:00 - 10:00	Introduction and basics
	10:00 - 10:30	Hands-on I
	10:30 - 10:45	Coffee break
	10:45 - 11:15	Hands-on II
	11:15 - 12:00	Hands-on III (Group Work)
	12:00 - 13:00	Lunch break
	13:00 - 14:00	Advanced concept and Convolutional Neural Network
	14:00 - 14:45	Hands-on IV
	14:45 - 15:15	Recap Day 1
29.05.2026 Day 2	09:00 - 10:00	Deep Generative Models
	10:00 - 10:45	Hands-on III
	10:45 - 11:15	Latest applications, code and knowledge sources
	11:15 - 11:30	Closing



We're here!

Cameras as main input for smart systems



From object recognition

to

traffic management system



<https://www.dlr.de/en/rm/research/expertise/3d-perception/classification>

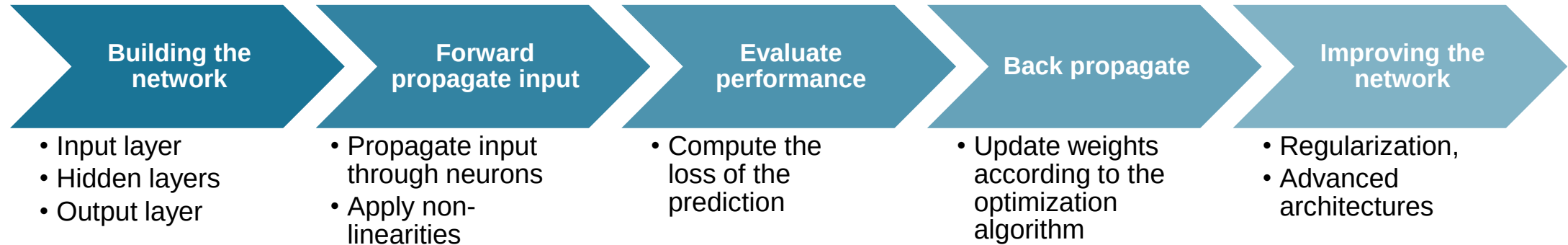
<https://www.dlr.de/en/eoc/about-us/remote-sensing-technology-institute/photogrammetry-and-image-analysis/public-datasets/xwheel>

II. Advanced Concept and Convolutional Neural Network

- Regularization
- Variants of Neural Networks
- Convolutional Neural Networks (CNN)

Inspired by lectures from Paris Saclay and MIT; images taken from these, if not noted otherwise

Neural network concepts



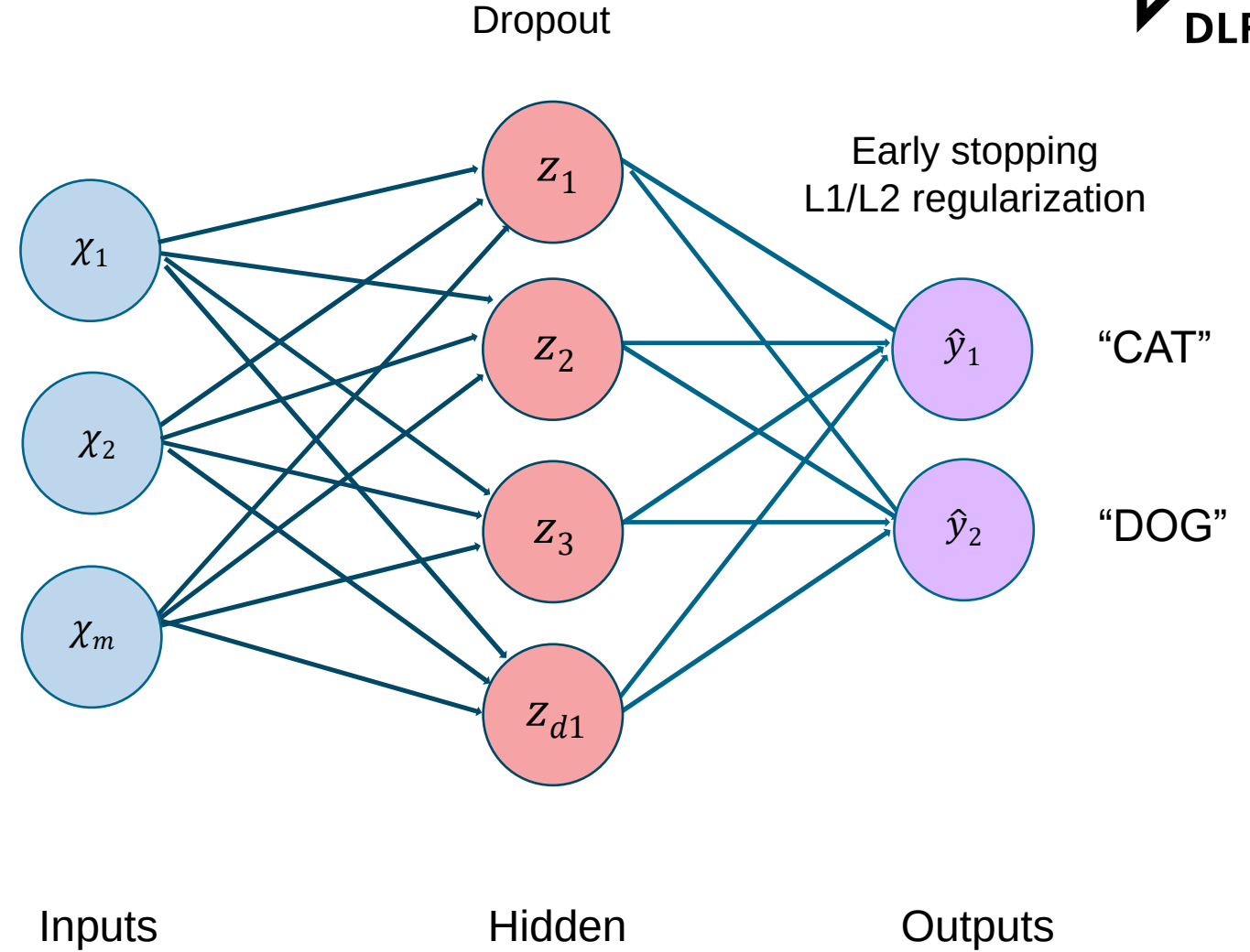
Improve the Network: Regularization

Data Augmentation



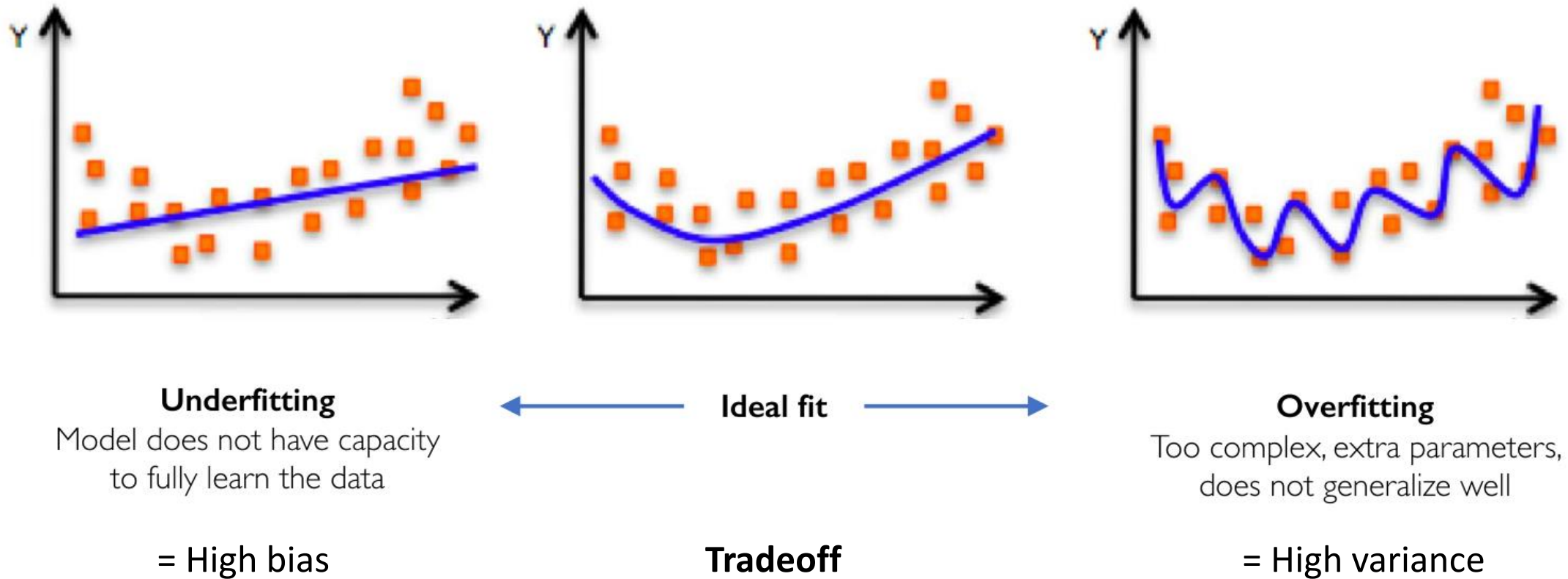
CAT

$$\begin{bmatrix} \chi_1 \\ \chi_2 \\ \chi_m \end{bmatrix}$$



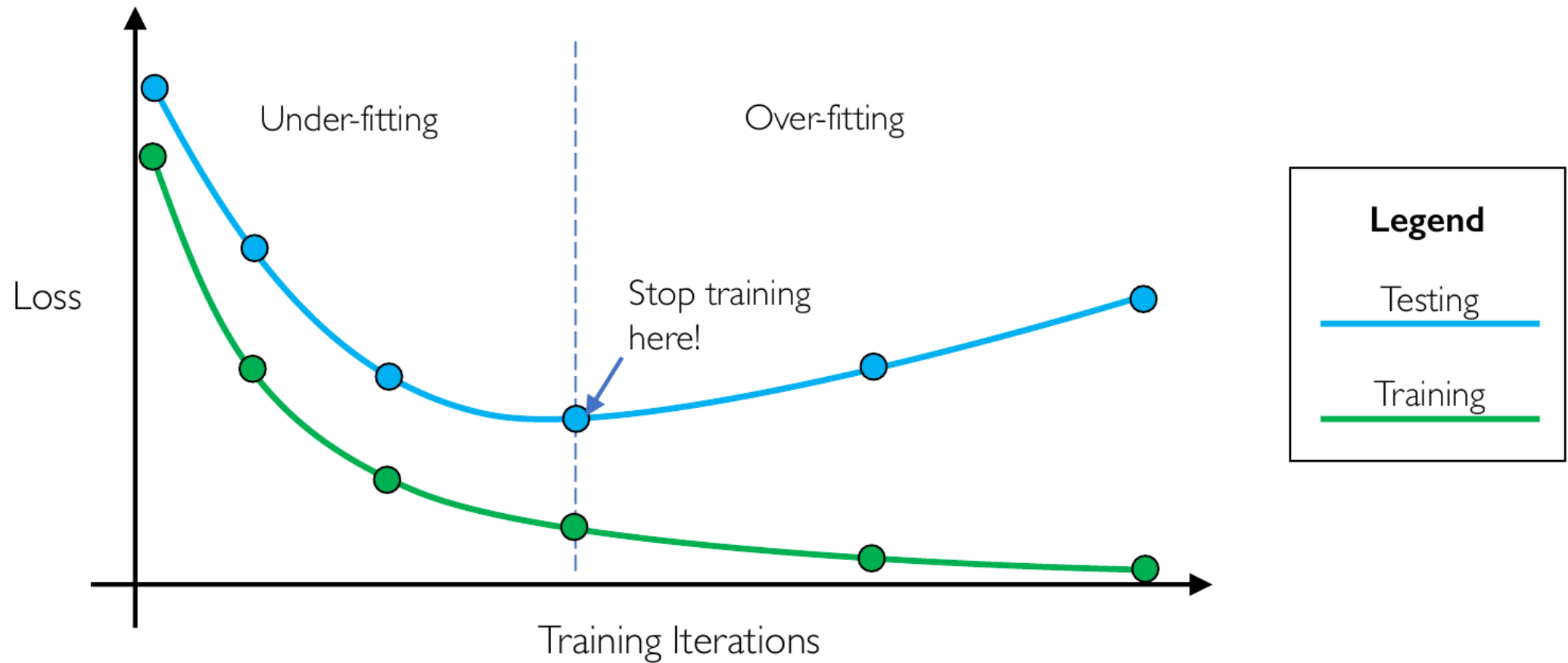
Regularization

The overfitting problem



Regularization

Preventing overfitting: Early stopping



Regularization

Preventing overfitting: l_1 regularization



We want to find the network weights that **achieve the lowest loss**

$$\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)}) + \frac{\lambda}{2} \sum_l |w_l|$$

→ Leads to sparser weights (more zeroes in weights) that are not too adapted to the data at hand

Regularization

Preventing overfitting: Data augmentation

Original



Flip



Random crop



Contrast



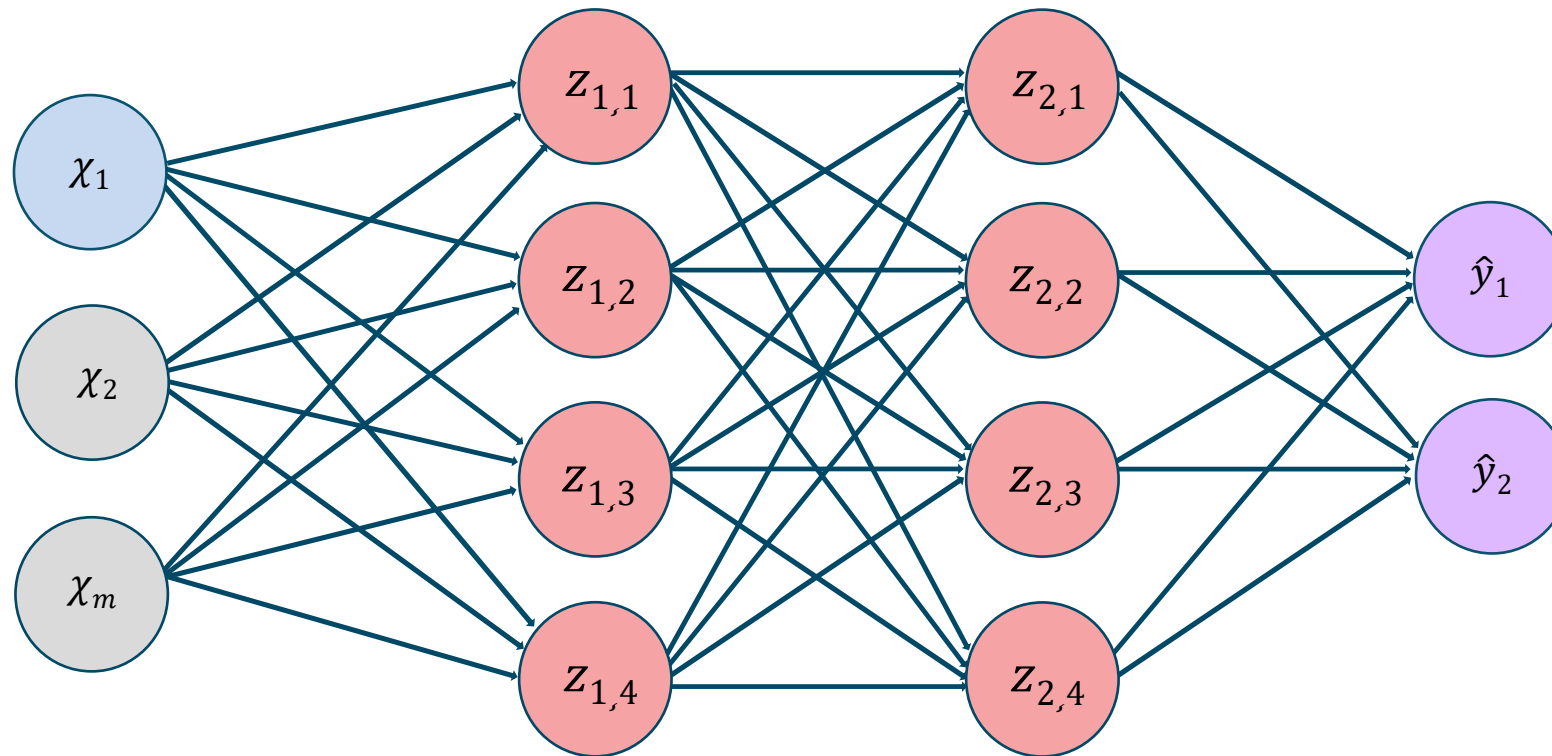
Tint



Regularization

Preventing overfitting: Dropout

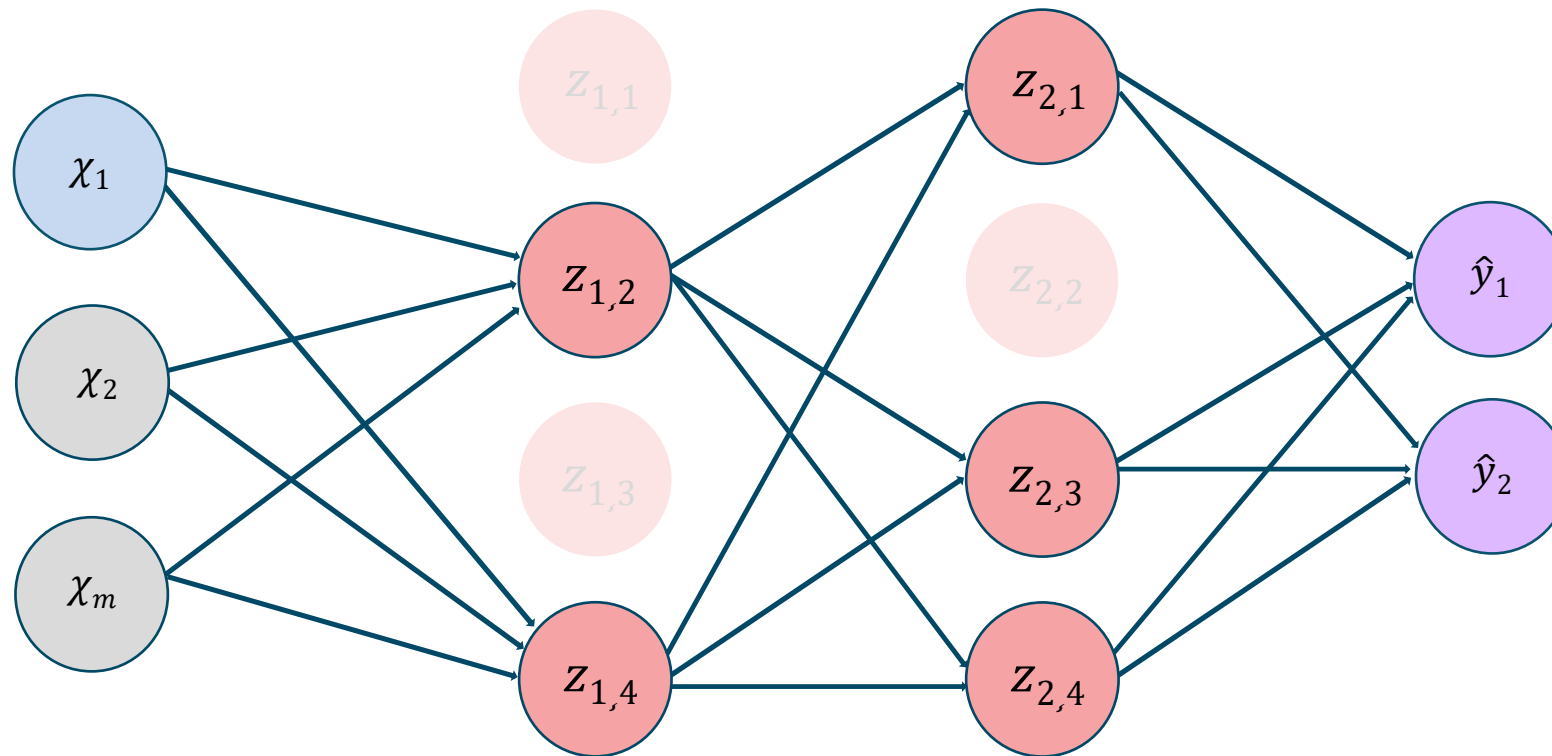
During training, randomly set some activations to 0



Regularization

Preventing overfitting: Dropout

During training, randomly set some activations to 0

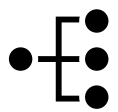


Other ways to improve the network



Data preprocessing

Data Cleaning (lack/noise)
Data Transformation (normalization, ...)
Data Reduction (aggregation, ...)



Network initialization

Random Initializations (e.g. Glorot)



Batch normalization

standardizes layer inputs to stabilize learning process & reducing training epochs



Optimizers

(Stochastic methods like Adam)



Training Schedules



Hyperparameters tuning



Minibatch composition

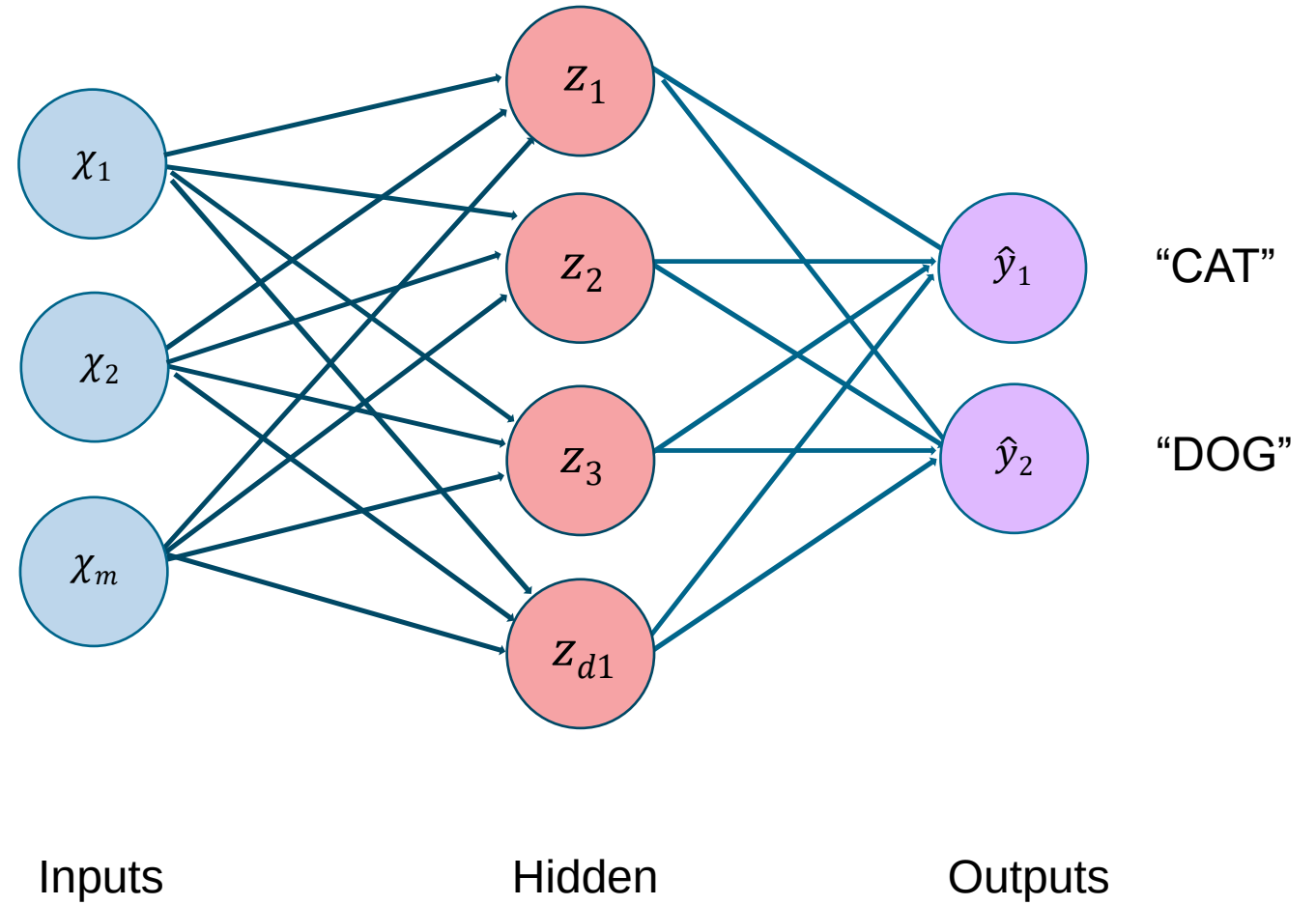
(shuffle data, take only a subset, ...)

Variants of Neural Networks



CAT

$$\begin{bmatrix} \chi_1 \\ \chi_2 \\ \chi_m \end{bmatrix}$$

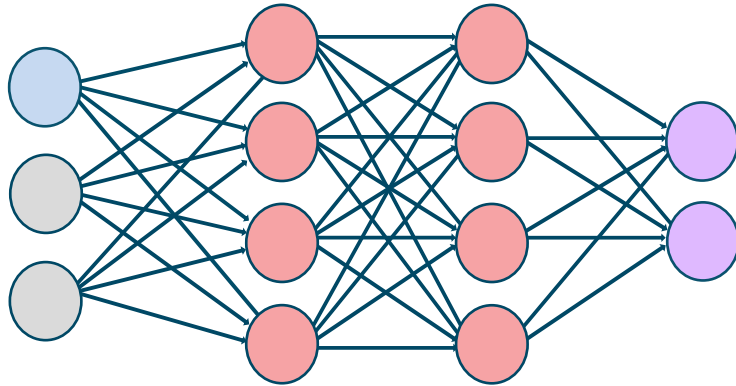


Types of Neural Networks Layer

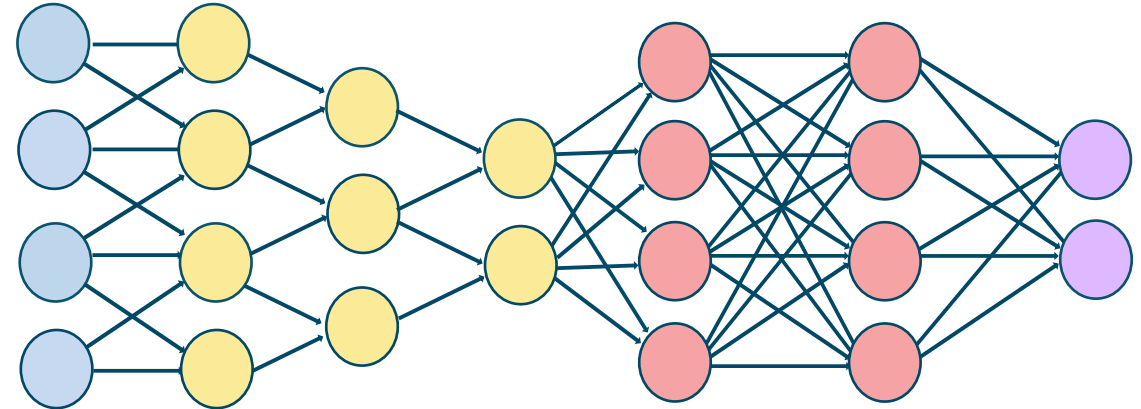
Application	Layer Type			
	Fully Connected (dense)	Convolution	Deconvolution	Recurrent
Image Classification	✓	✓		
Image segmentation		✓	✓	
Text processing	✓			✓
Speech recognition	✓			✓
Time series	✓			✓
Image-to-image translation (GAN)	✓	✓	✓	
Autoencoders	✓	✓		
Deepfake	✓	✓		✓

Variants of Neural Network - Examples

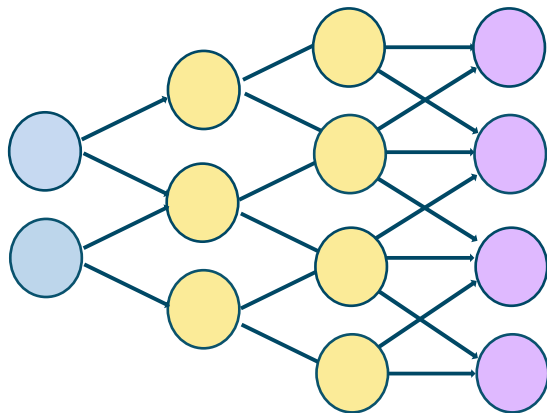
Deep Feed Forward (DFF) (fully connected)



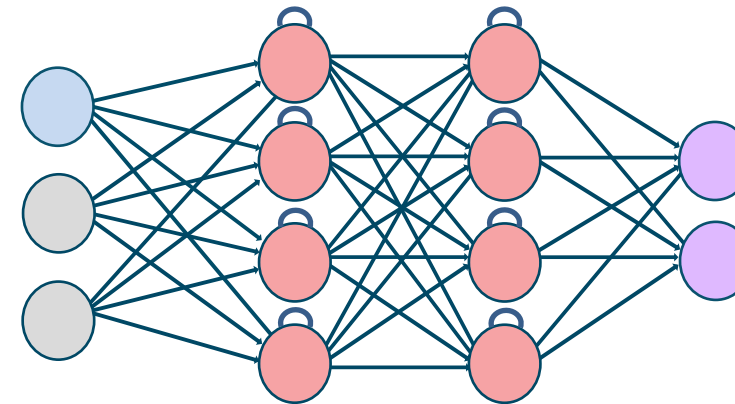
Deep convolution network (DCN)



Deconvolution network (DN)

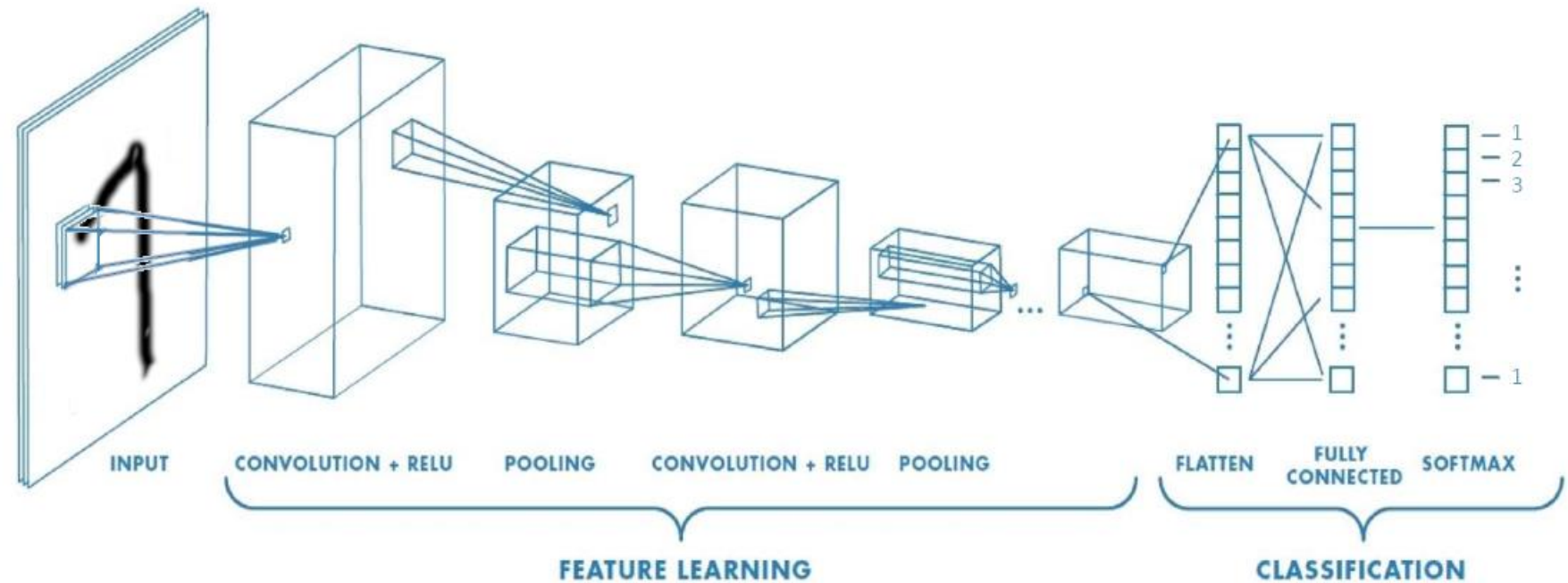


Recurrent neural network (RNN)

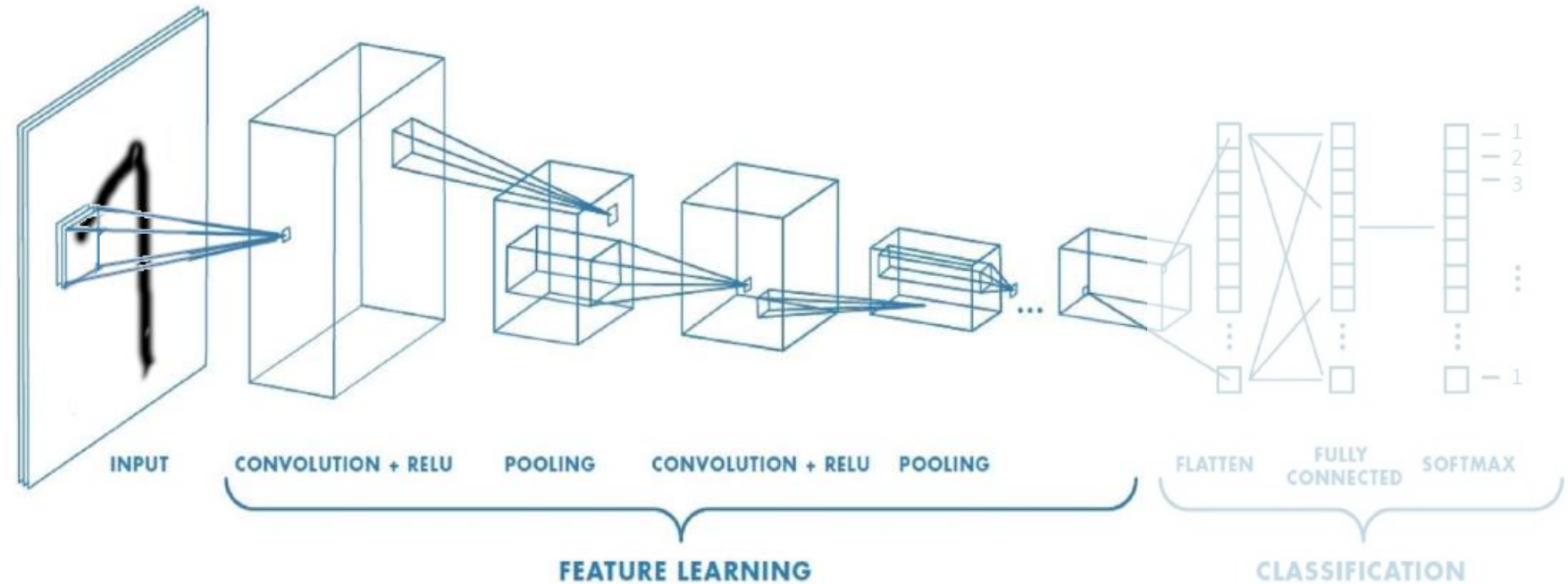


● = Input
● = Output

Convolutional Neural Networks



Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**

Convolutional Neural Networks

Feature extraction with convolutions



Original



Sharpen



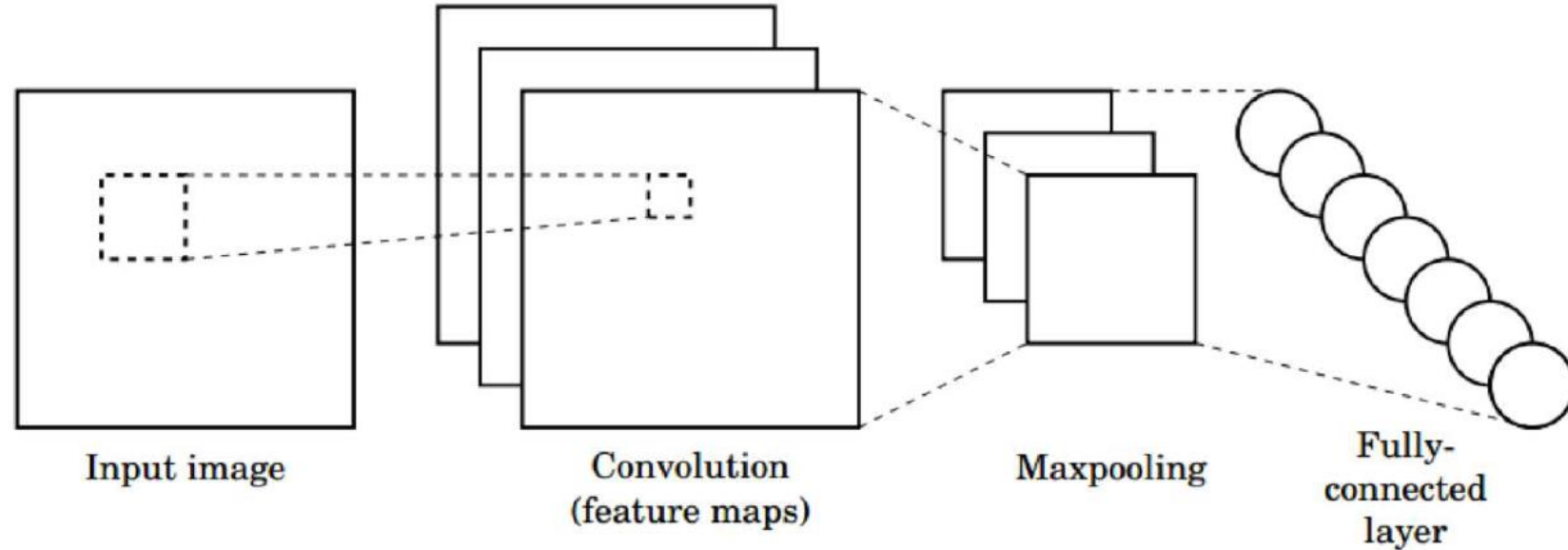
Edge Detect



“Strong” Edge
Detect

Convolutional Neural Networks

Building a CNN



1. **Convolution:** Apply filters with learned weights to generate feature maps.
2. **Non-linearity:** Often ReLU.
3. **Pooling:** Downsampling operation on each feature map.

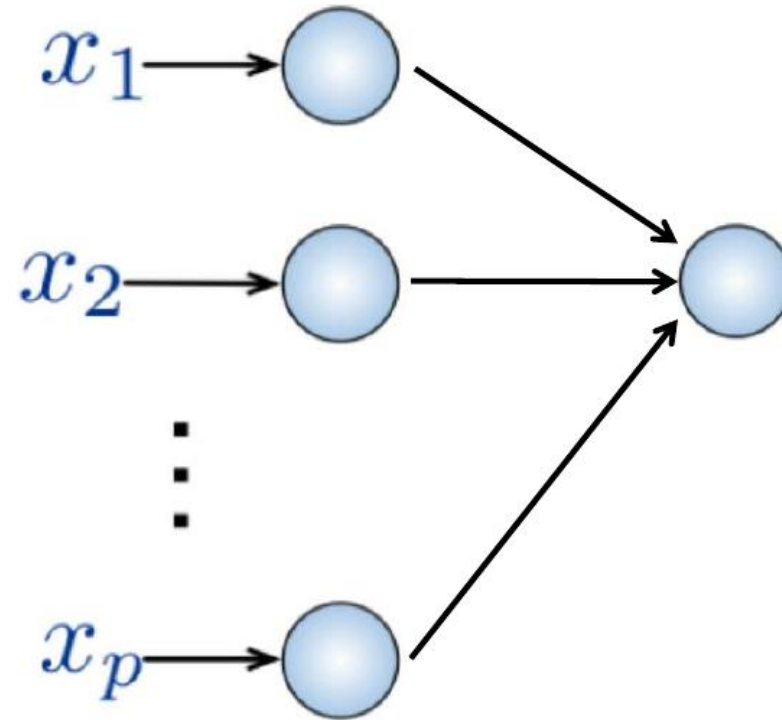
Train model with image data.
Learn weights of filters in convolutional layers.

Convolutional Neural Networks

Learning on image data

Input:

- 2D image
- Vector of pixel values



Fully Connected:

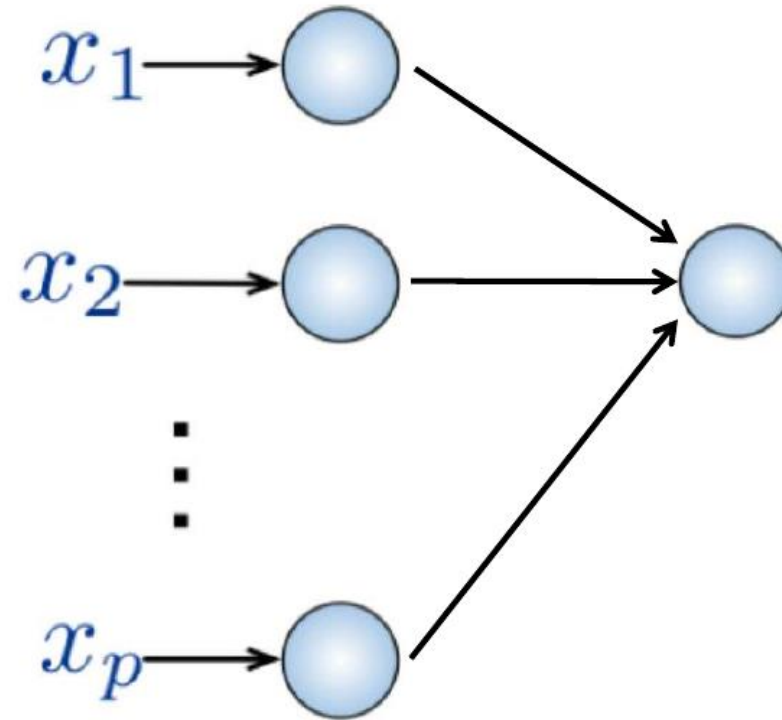
- Connect neuron in hidden layer to all neurons in input layer
- No spatial information!
- And many, many parameters!

Convolutional Neural Networks

Learning on image data

Input:

- 2D image
- Vector of pixel values



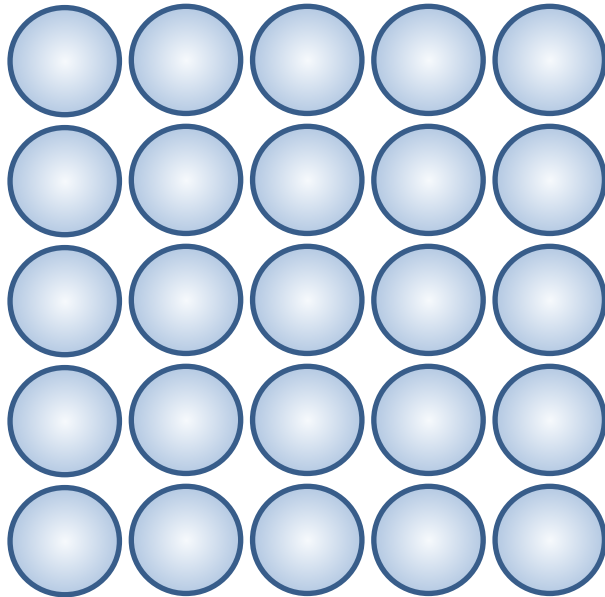
Fully Connected:

- Connect neuron in hidden layer to all neurons in input layer
- No spatial information!
- And many, many parameters!

How can we use **spatial structure** in the input to inform the architecture of the network?

Convolutional Neural Networks

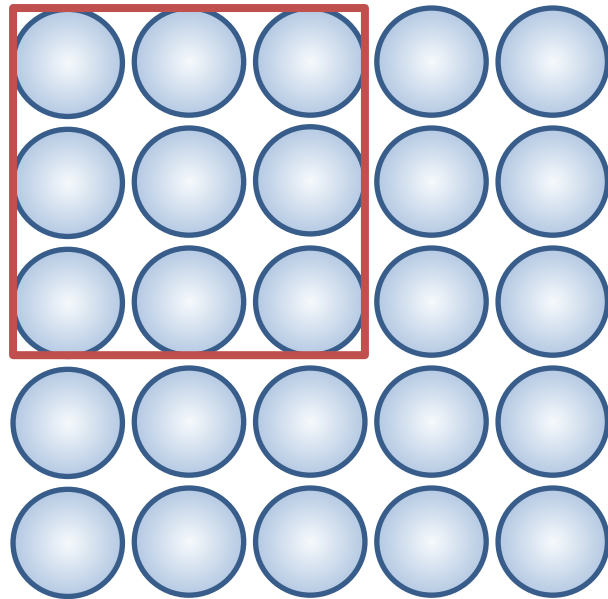
Learning on image data



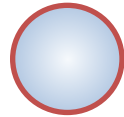
Input: 2D image (5x5 or 256x256 or ...)
Array of pixel values

Convolutional Neural Networks

Learning on image data

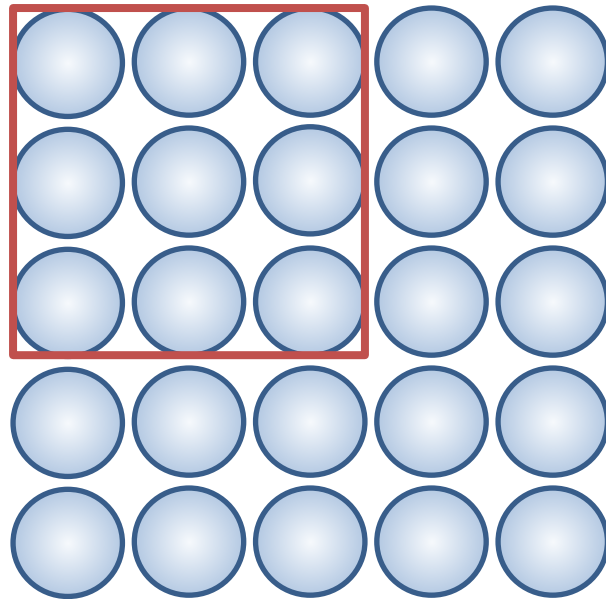


Idea: connect patches of input to neurons in hidden layer
Neuron connected to the region of input only „sees“ these values.



Convolutional Neural Networks

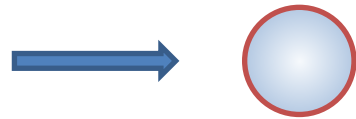
Learning on image data



3x3 filter: matrix
of weights W_{ij}

For a neuron in hidden layer:

- Take inputs from patch
- Compute weighted sum
- Apply bias



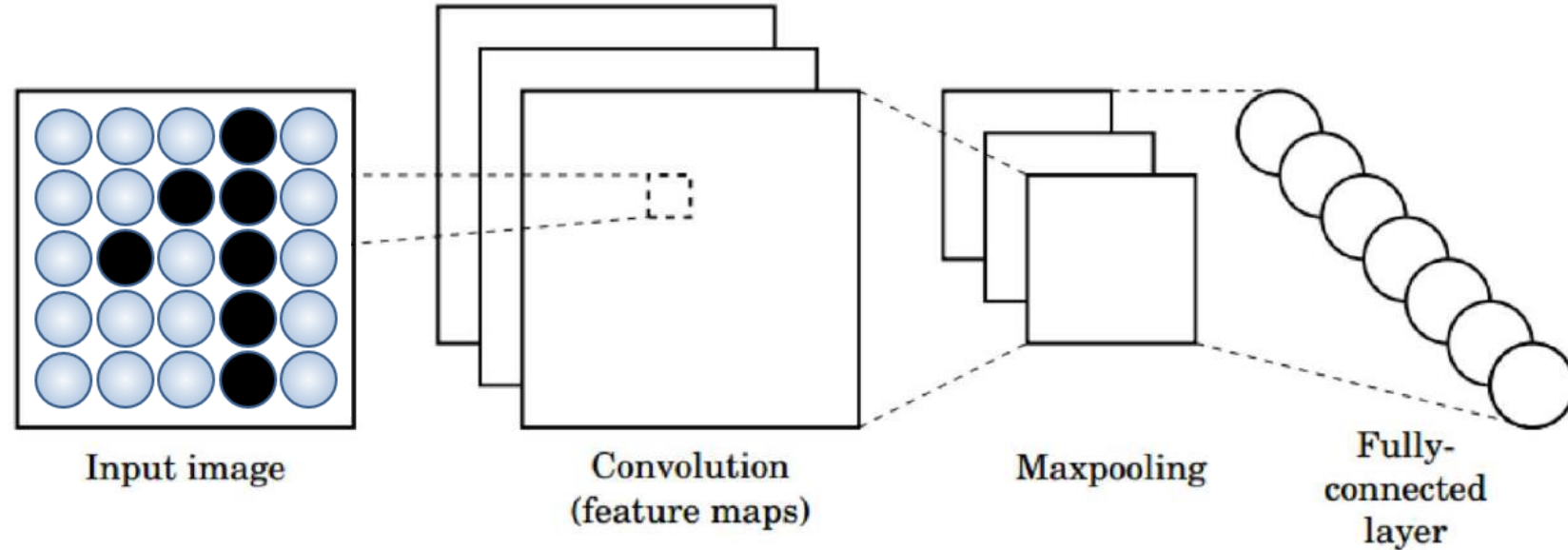
$$\sum_{i=1}^3 \sum_{j=1}^3 W_{ij} X_{i+p,j+q} + b$$

For neuron (p,q) in hidden layer

- 1) Applying a window of weights
- 2) Computing linear combinations
- 3) Activating with non-linear function

Convolutional Neural Networks

Building a CNN - Example

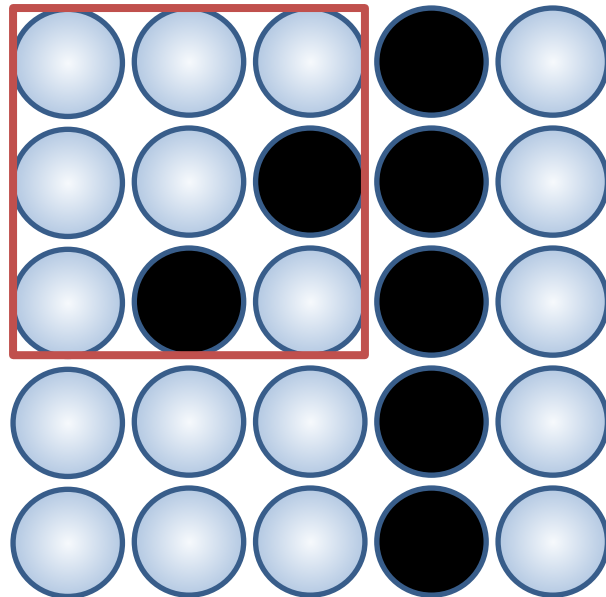
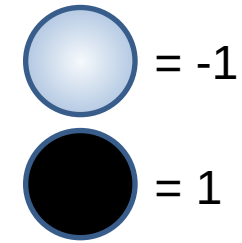


1. **Convolution:** Apply filters with learned weights to generate feature maps.
2. **Non-linearity:** Often ReLU.
3. **Pooling:** Downsampling operation on each feature map.

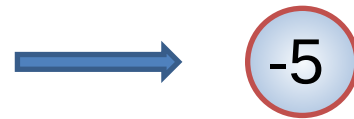
Train model with image data.
Learn weights of filters in convolutional layers.

Convolutional Neural Networks

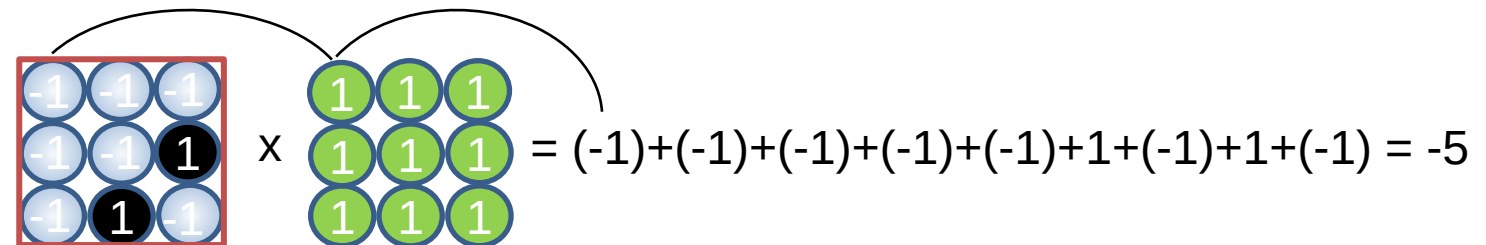
Learning on image data - Example



- Connect patch in input layer to a single neuron in subsequent layer
- Use a **sliding window** to define connections
- Ex: extracting feature with filter fi_0

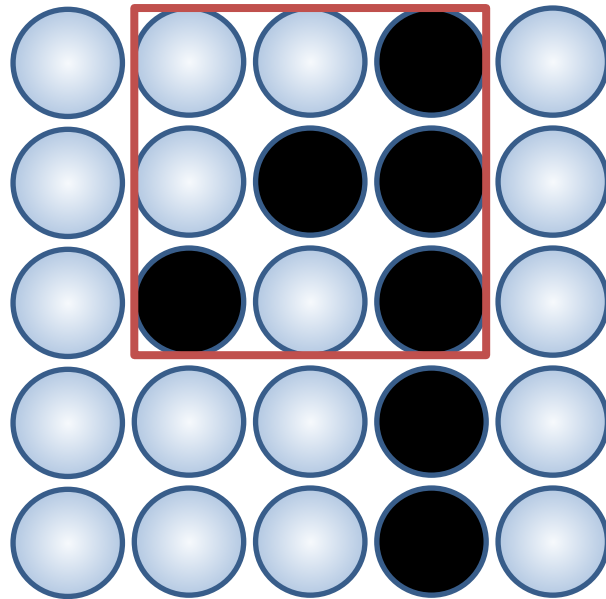
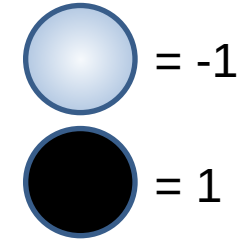
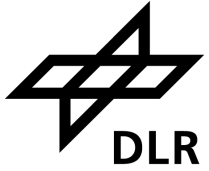


Multiply same indices of clipping and filter and sum it up.

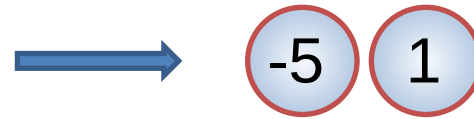

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & -1 & 1 \\ -1 & 1 & -1 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = (-1)+(-1)+(-1)+(-1)+(-1)+1+(-1)+1+(-1) = -5$$

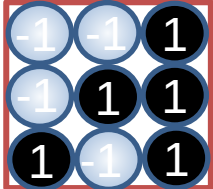
Convolutional Neural Networks

Learning on image data - Example



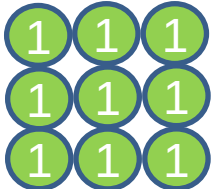
- **Step size** here is one and the clipping is 3x3





A 3x3 grid of circles with values: top row [-1, -1, 1], middle row [-1, 1, 1], bottom row [1, -1, 1]. A red box highlights this grid.

x

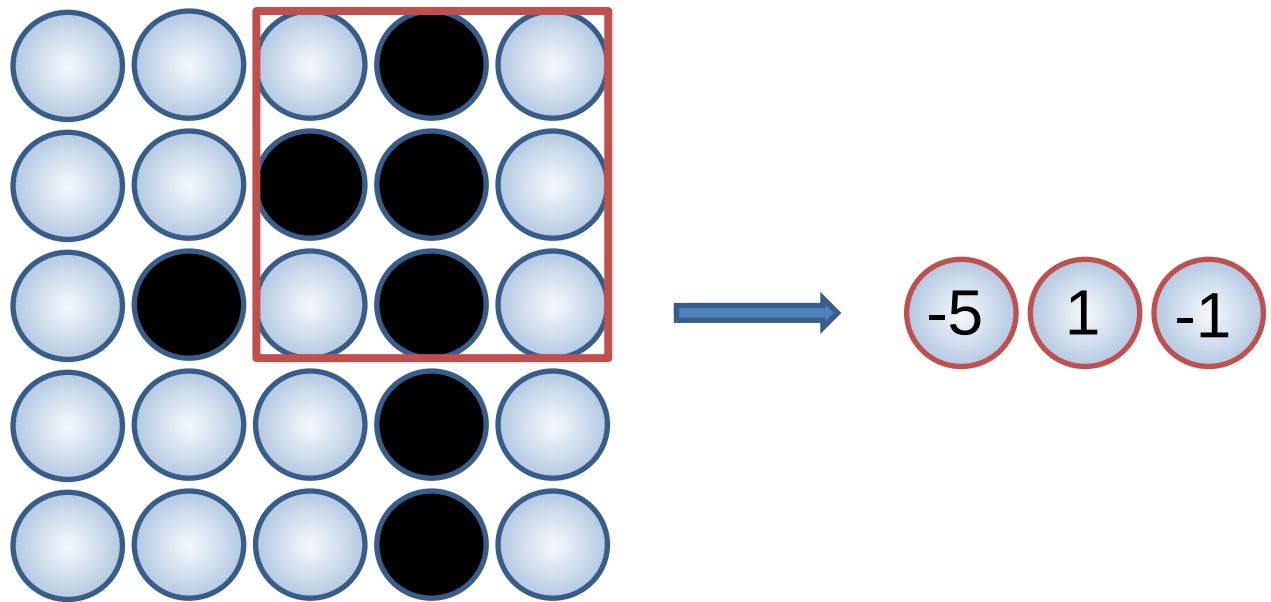
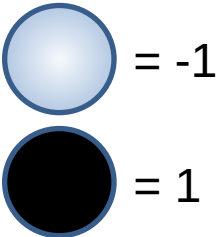


A 3x3 grid of green circles, all containing the value 1.

= 1

Convolutional Neural Networks

Learning on image data - Example



Kernel operation:

-1	1	-1
1	1	-1
-1	1	-1

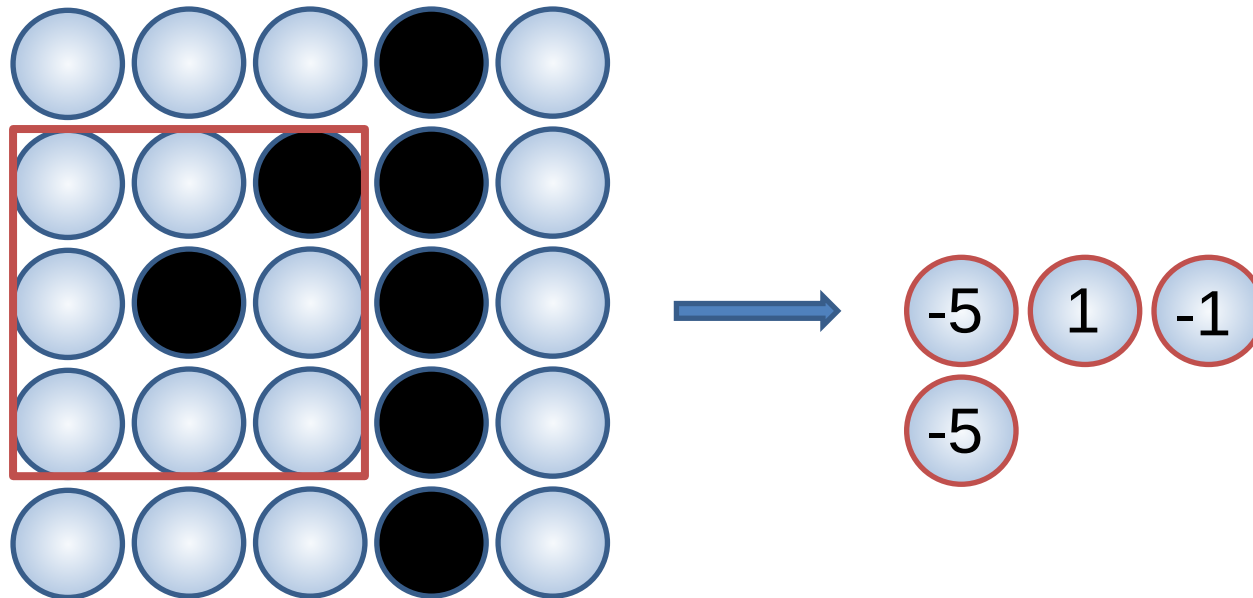
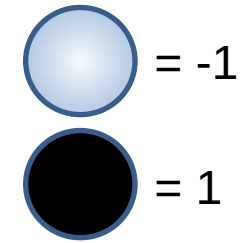
 $\times$

1	1	1
1	1	1
1	1	1

 $= -1$

Convolutional Neural Networks

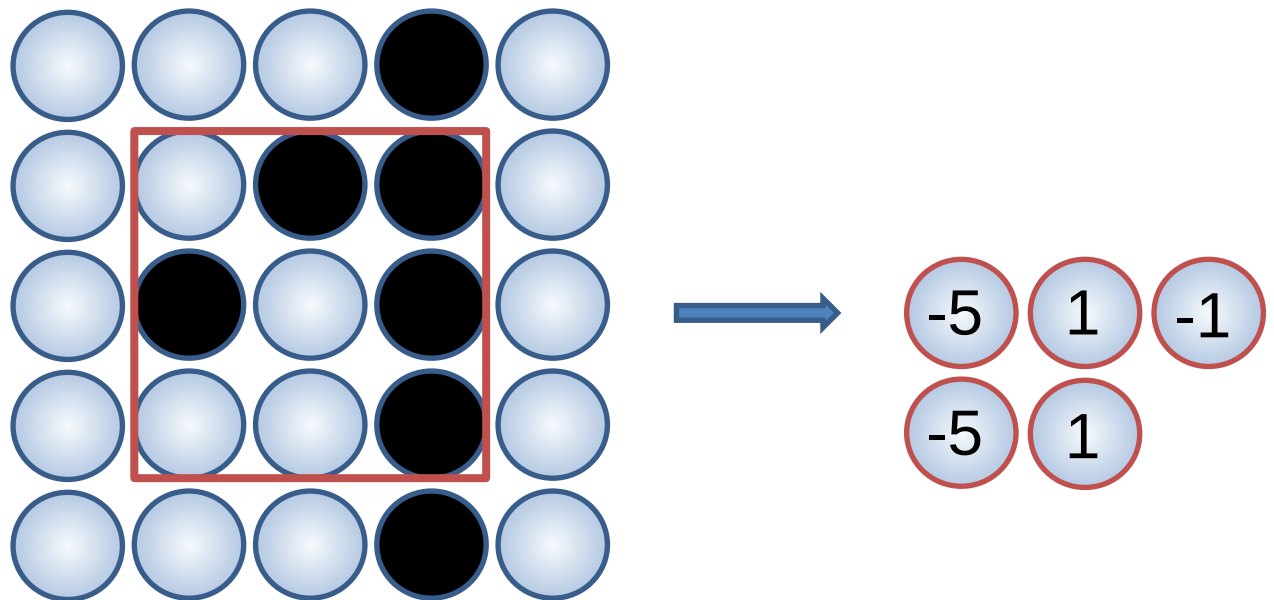
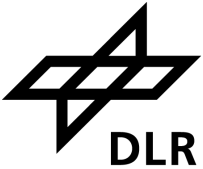
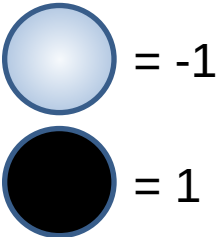
Learning on image data - Example



$$\boxed{\dots} \times \begin{matrix} \textcircled{1} & \textcircled{1} & \textcircled{1} \\ \textcircled{1} & \textcircled{1} & \textcircled{1} \\ \textcircled{1} & \textcircled{1} & \textcircled{1} \end{matrix} = -5$$

Convolutional Neural Networks

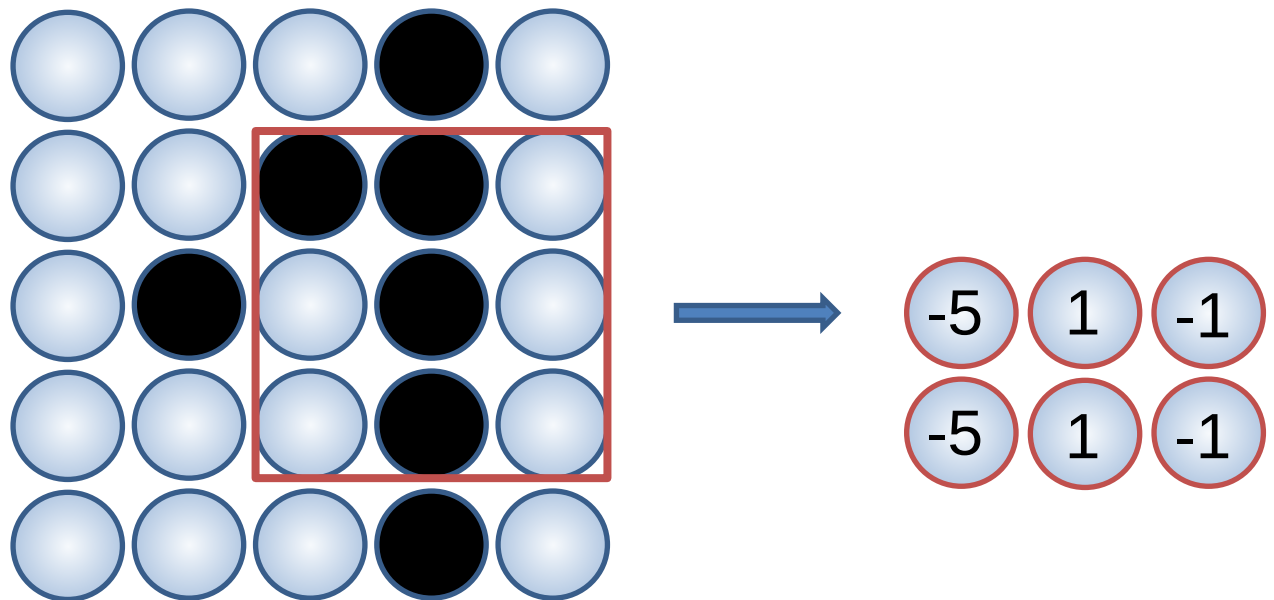
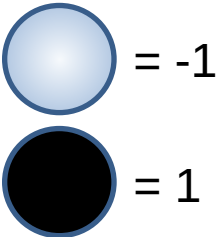
Learning on image data - Example

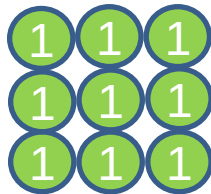


$$\boxed{\dots} \times \begin{matrix} \textcircled{1} & \textcircled{1} & \textcircled{1} \\ \textcircled{1} & \textcircled{1} & \textcircled{1} \\ \textcircled{1} & \textcircled{1} & \textcircled{1} \end{matrix} = 1$$

Convolutional Neural Networks

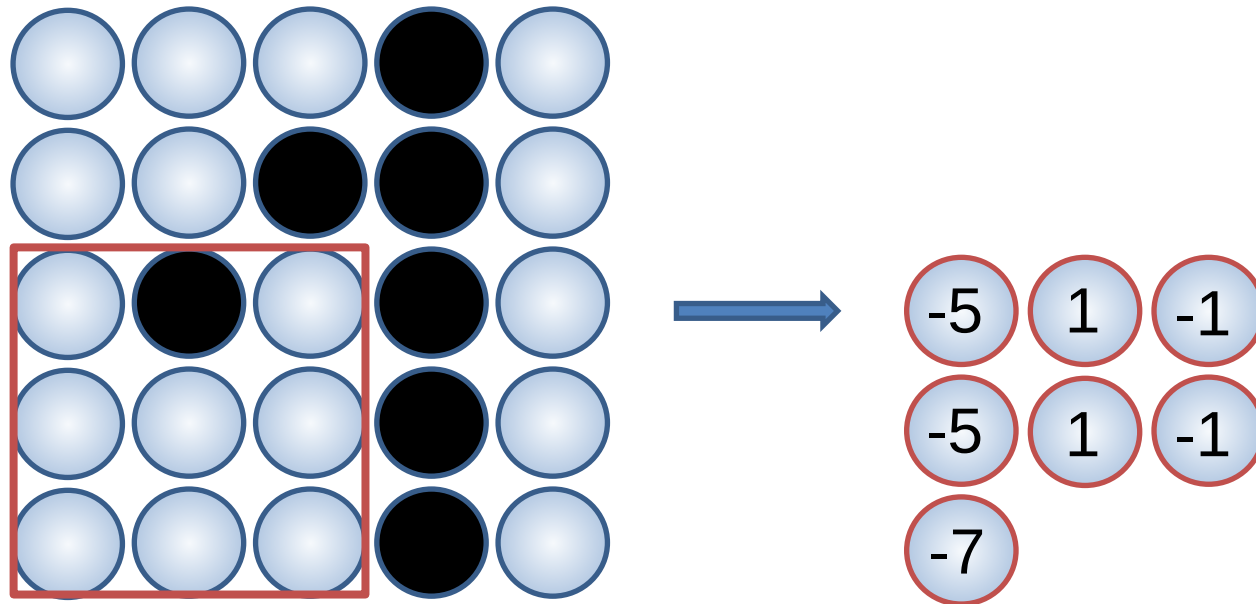
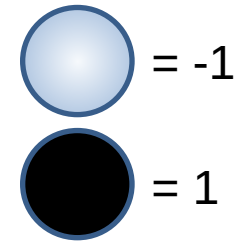
Learning on image data - Example

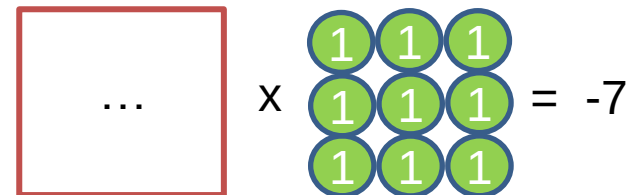


 ... $\times$  = -1

Convolutional Neural Networks

Learning on image data - Example

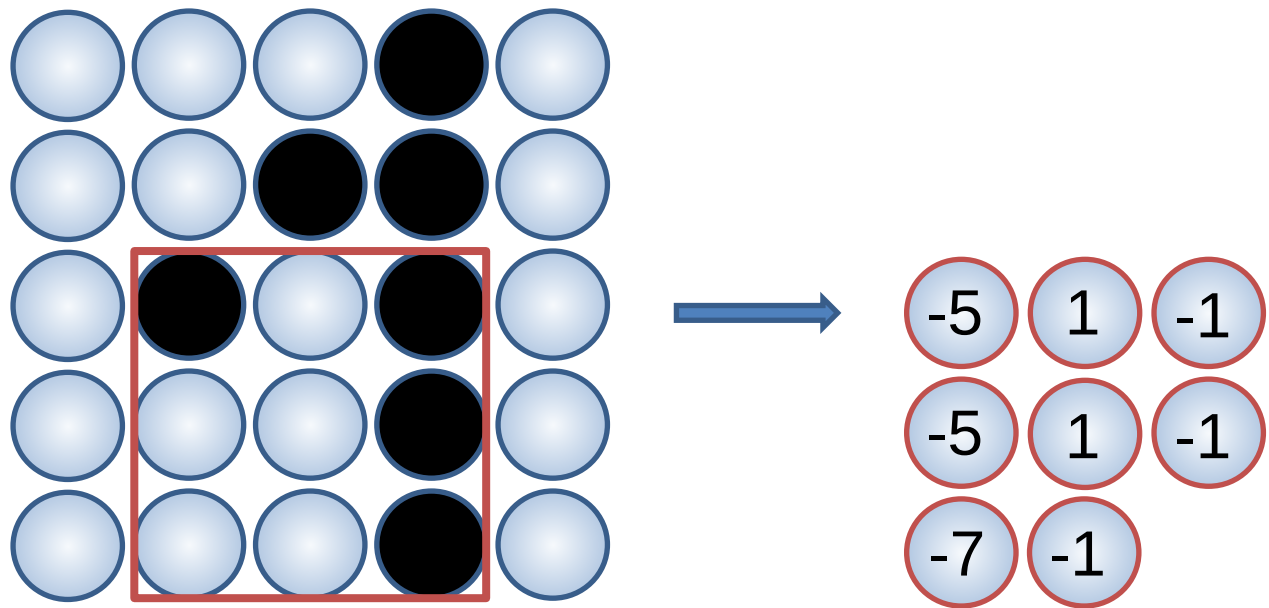
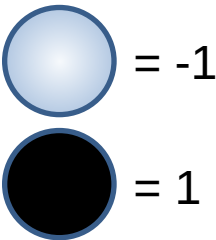


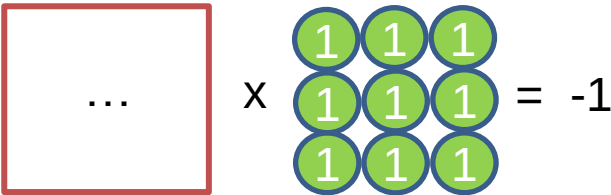


$\dots \times \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix} = -7$

Convolutional Neural Networks

Learning on image data - Example

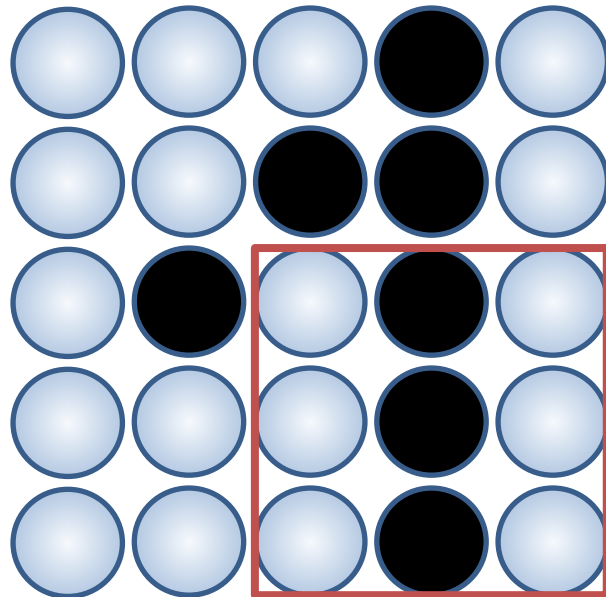
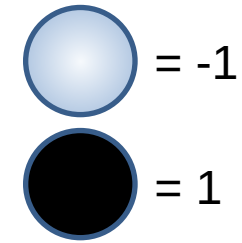
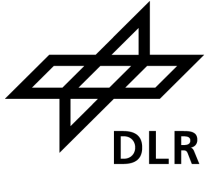




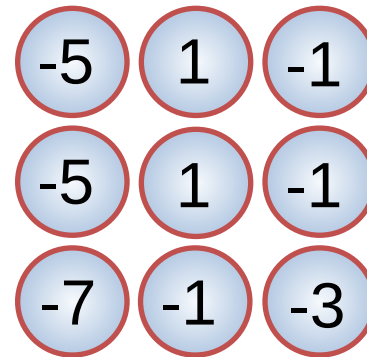
$\dots \times \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix} = -1$

Convolutional Neural Networks

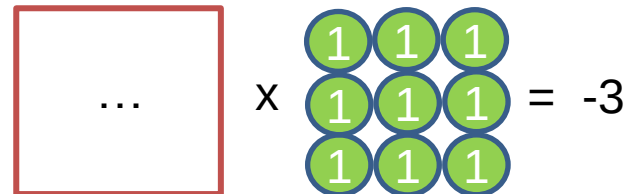
Learning on image data - Example



- This results in a "feature map" → here : 9 neurons for the 5x5 image

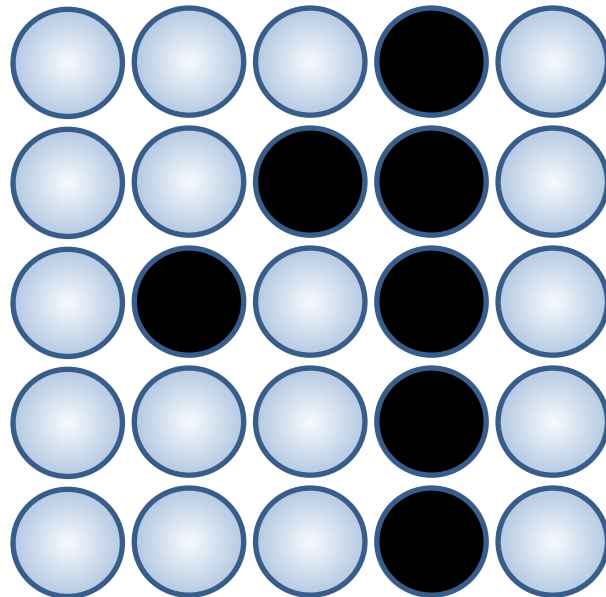
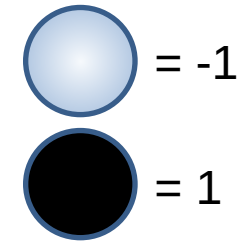


- Filter of size 3x3 → 9 separate weights
- Scan step wise the image with the filter
- That's **Convolution**



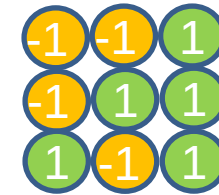
Convolutional Neural Networks

Learning – Feature Extraction



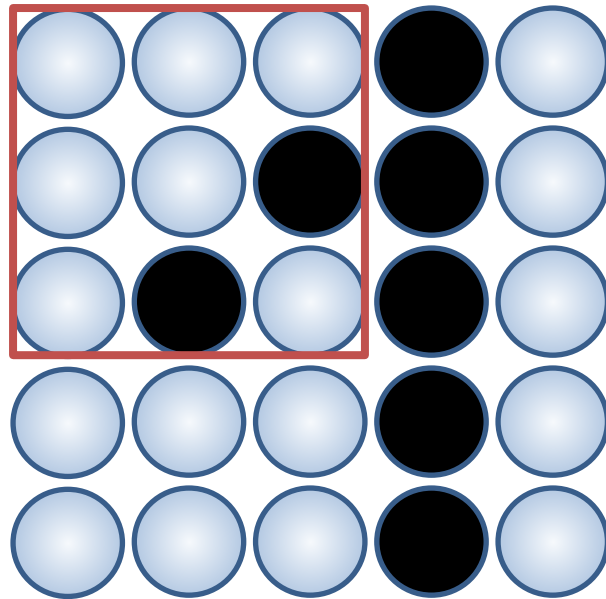
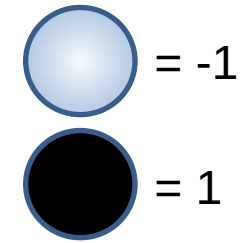
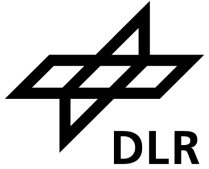
- Repeat the scan process with **multiple filters**

Next: extracting feature with filter $f_{i_1} =$

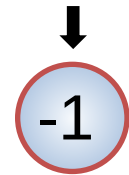


Convolutional Neural Networks

Learning – Feature Extraction



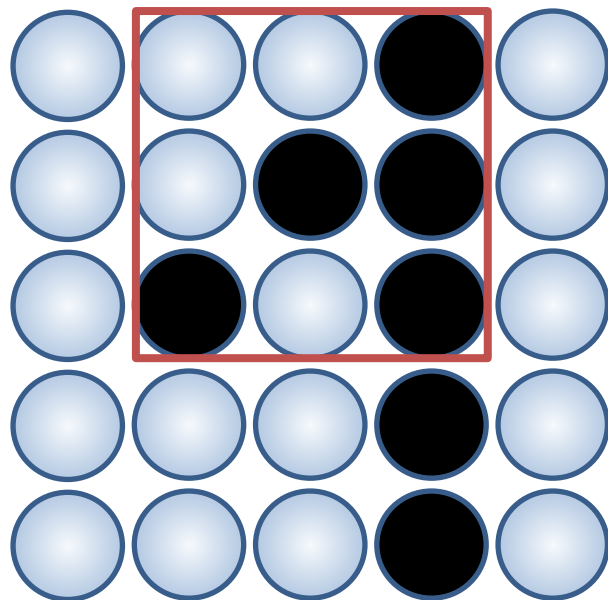
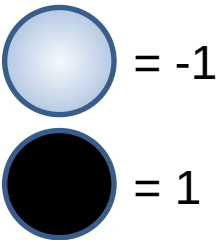
Low score !
barely matches



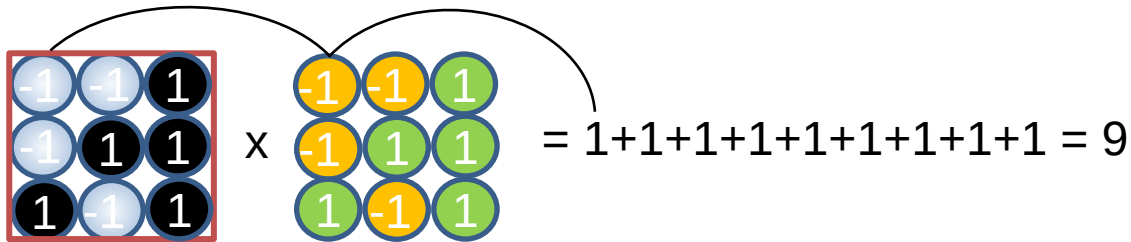
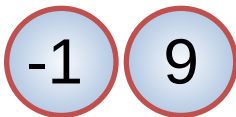
$$\begin{array}{|c|c|c|} \hline -1 & -1 & -1 \\ \hline -1 & -1 & 1 \\ \hline -1 & 1 & -1 \\ \hline \end{array} \times \begin{array}{|c|c|c|} \hline -1 & -1 & 1 \\ \hline -1 & 1 & 1 \\ \hline 1 & -1 & 1 \\ \hline \end{array} = 1+1+(-1)+1+(-1)+1+(-1)+(-1)+(-1) = -1$$

Convolutional Neural Networks

Learning – Feature Extraction

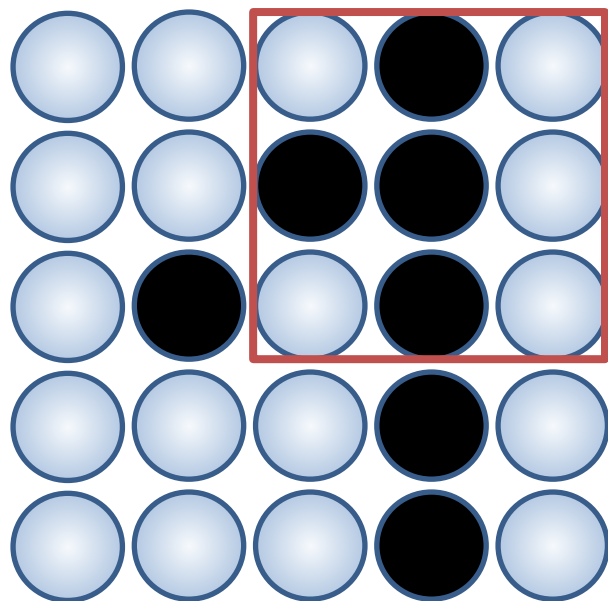
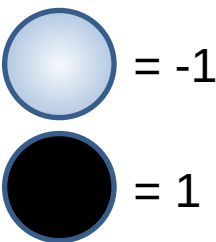


Maximum score !
Filter matches full

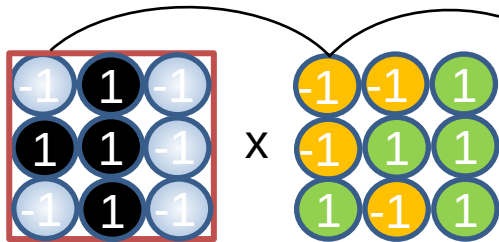


Convolutional Neural Networks

Learning – Feature Extraction



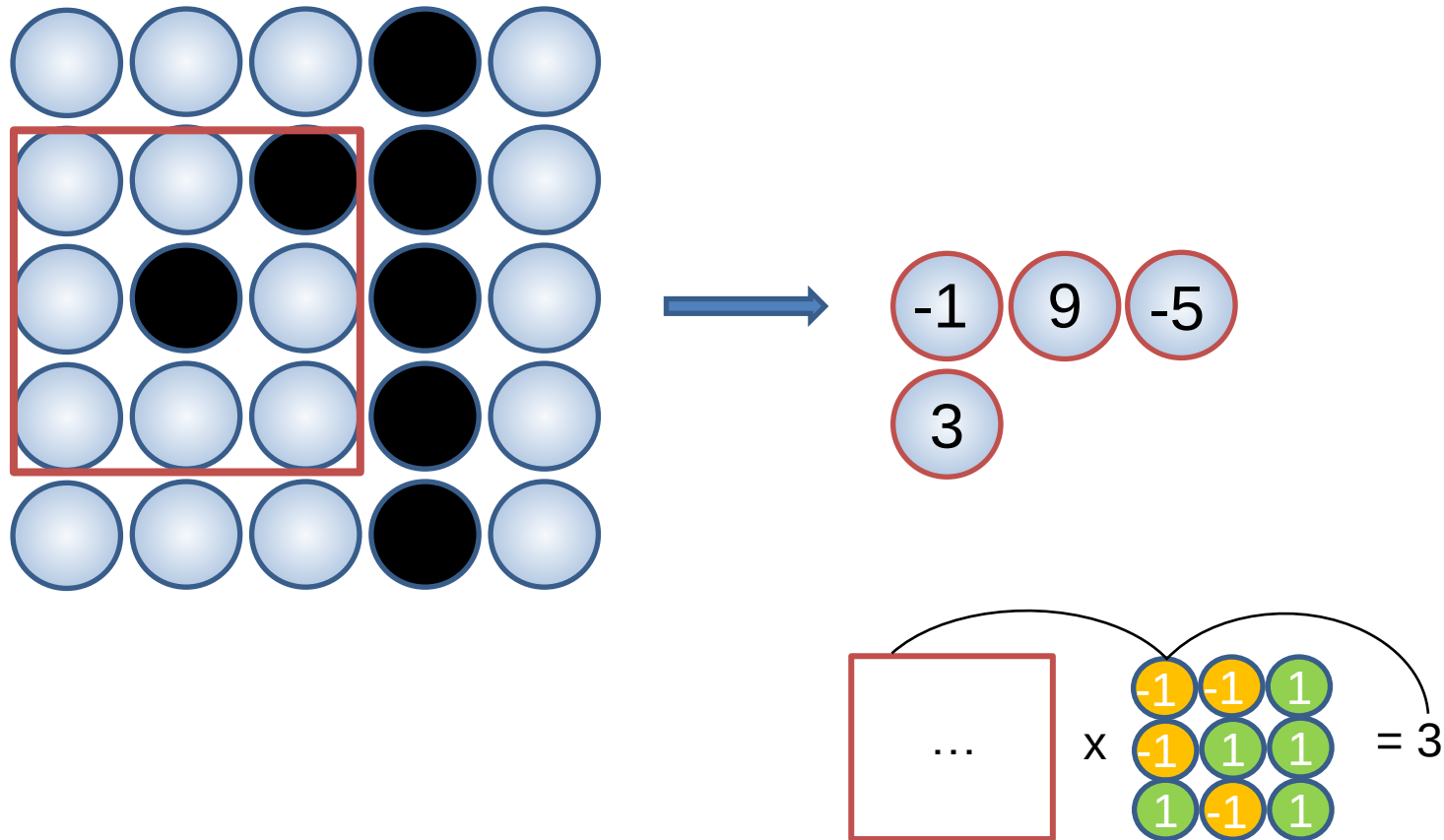
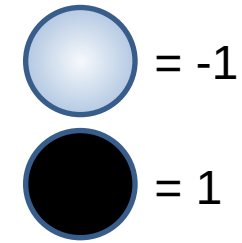
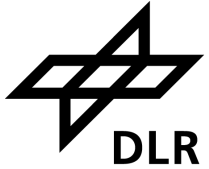
Very low score !
Many mismatches



$$= 1 + (-1) + (-1) + (-1) + 1 + (-1) + (-1) + (-1) + (-1) = -5$$

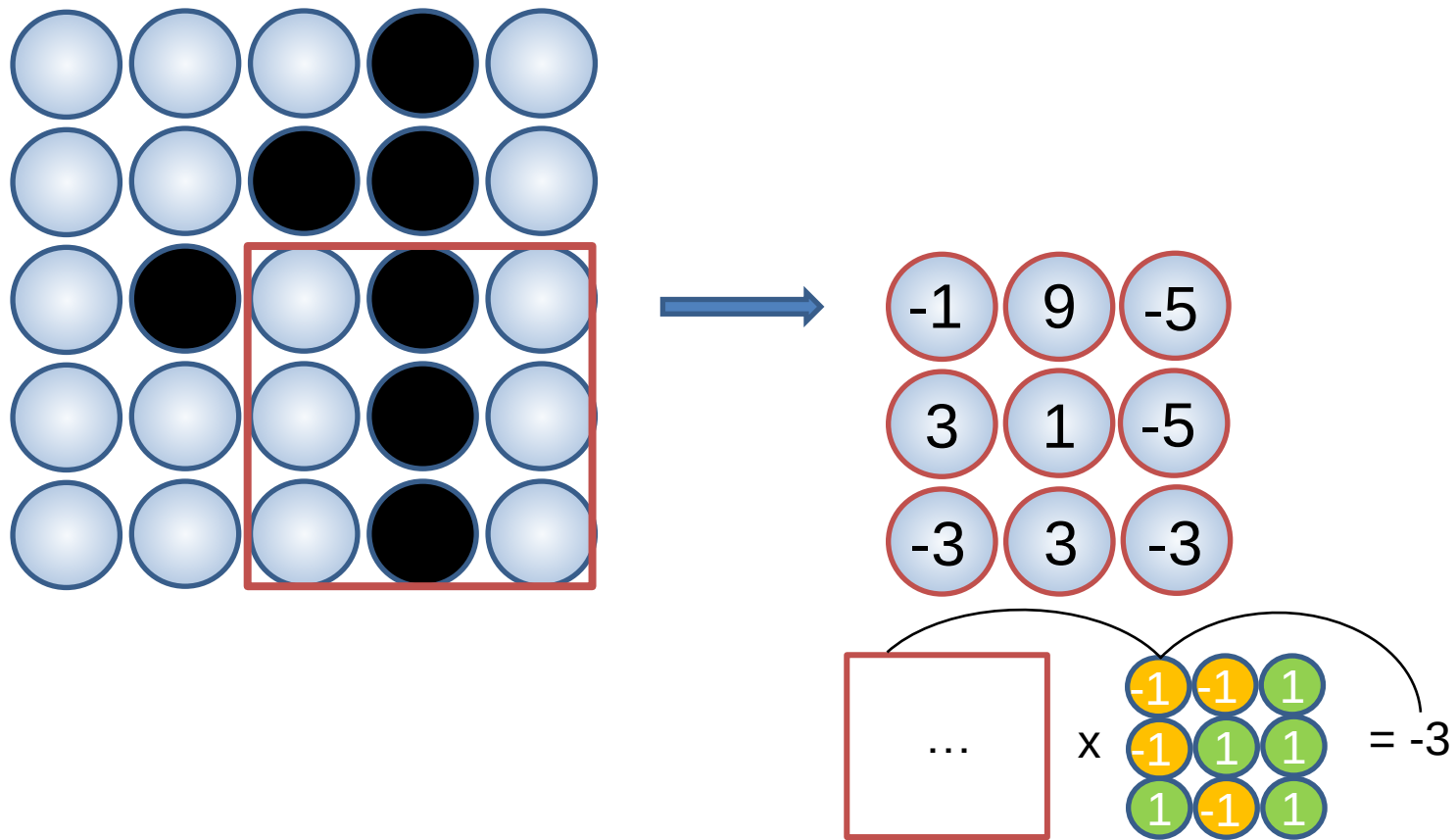
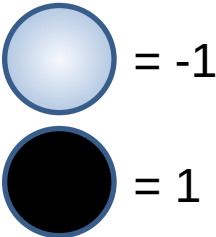
Convolutional Neural Networks

Learning – Feature Extraction



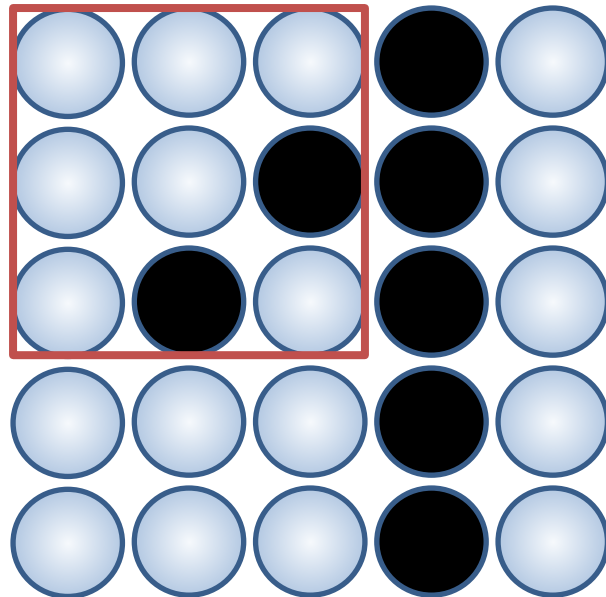
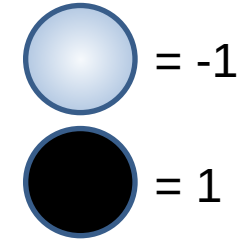
Convolutional Neural Networks

Learning – Feature Extraction



Convolutional Neural Networks

Learning – Feature Extraction

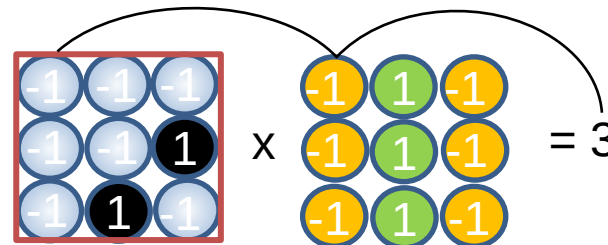
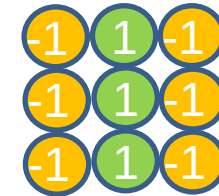


- Different feature will extract different features
- **Spatially share** parameters of each filter



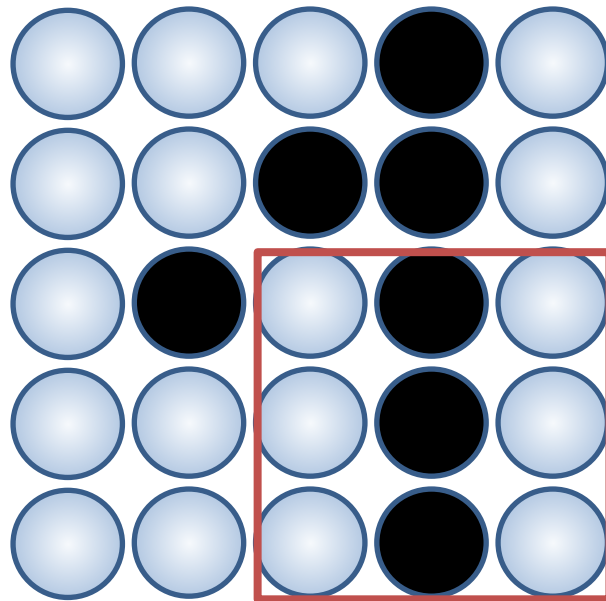
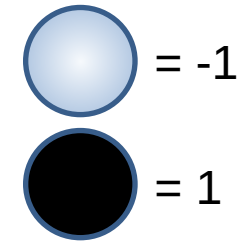
3

Next filter $\mathbf{f}_2 =$

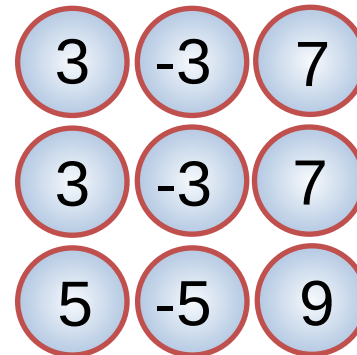


Convolutional Neural Networks

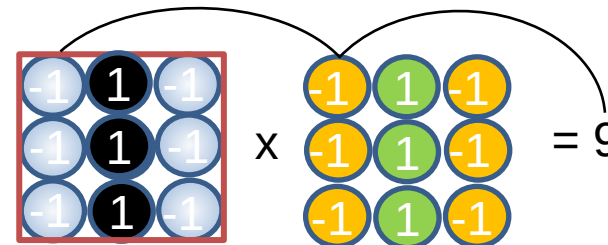
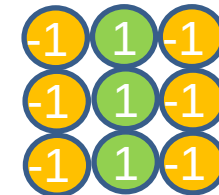
Learning – Feature Extraction



- Different feature will extract different features
- **Spatially share** parameters of each filter

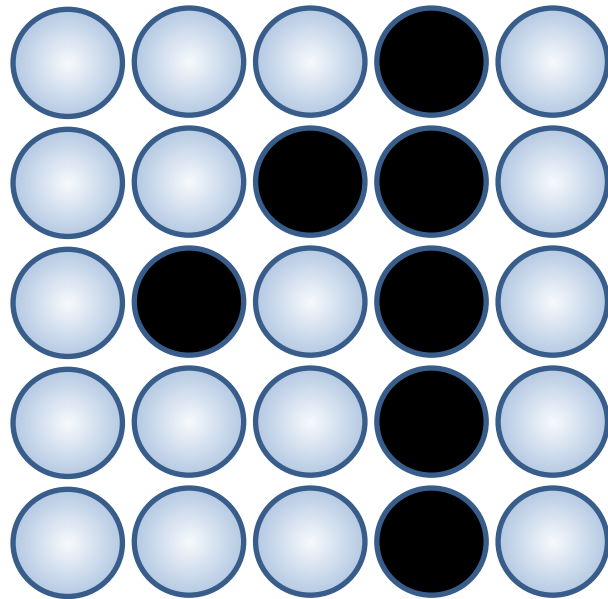
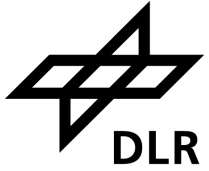


Next filter $\mathbf{f}_2 =$



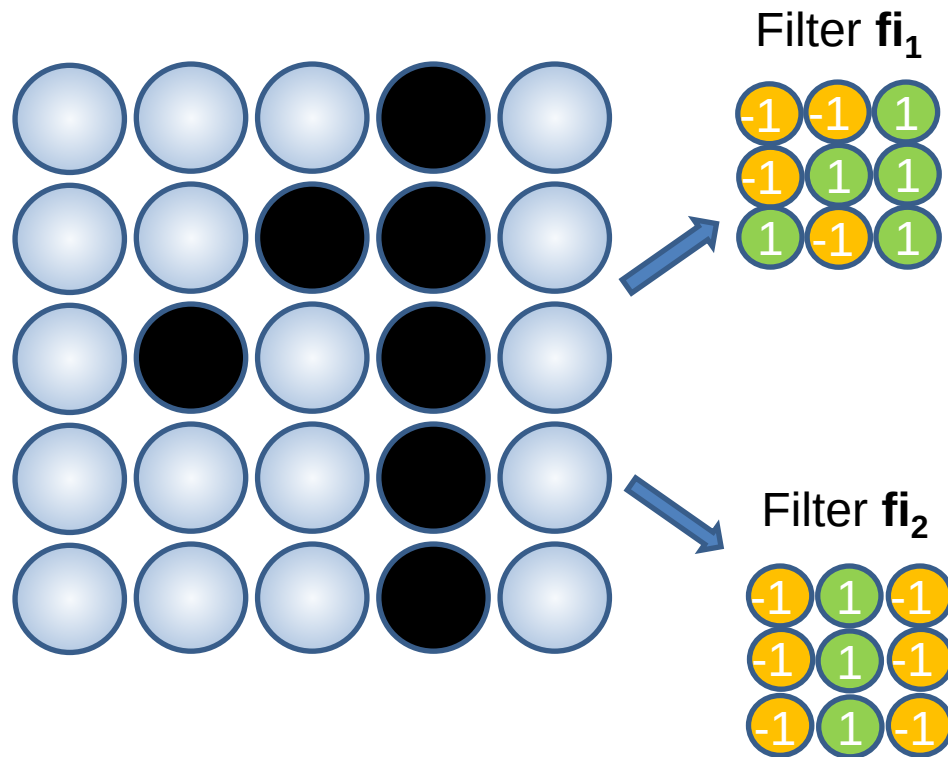
Convolutional Neural Networks

Learning – Feature Extraction: filter comparison



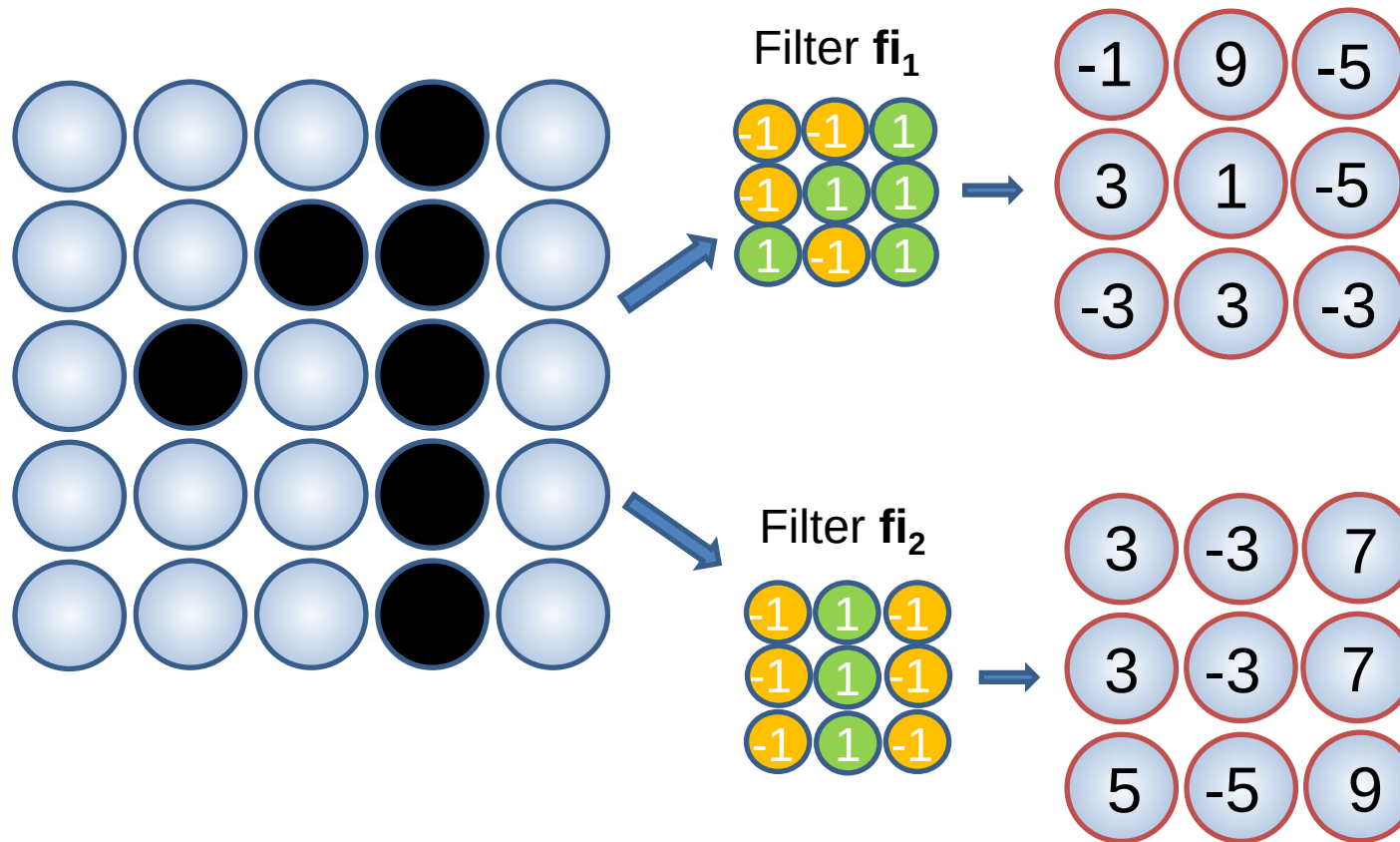
Convolutional Neural Networks

Learning – Feature Extraction: filter comparison



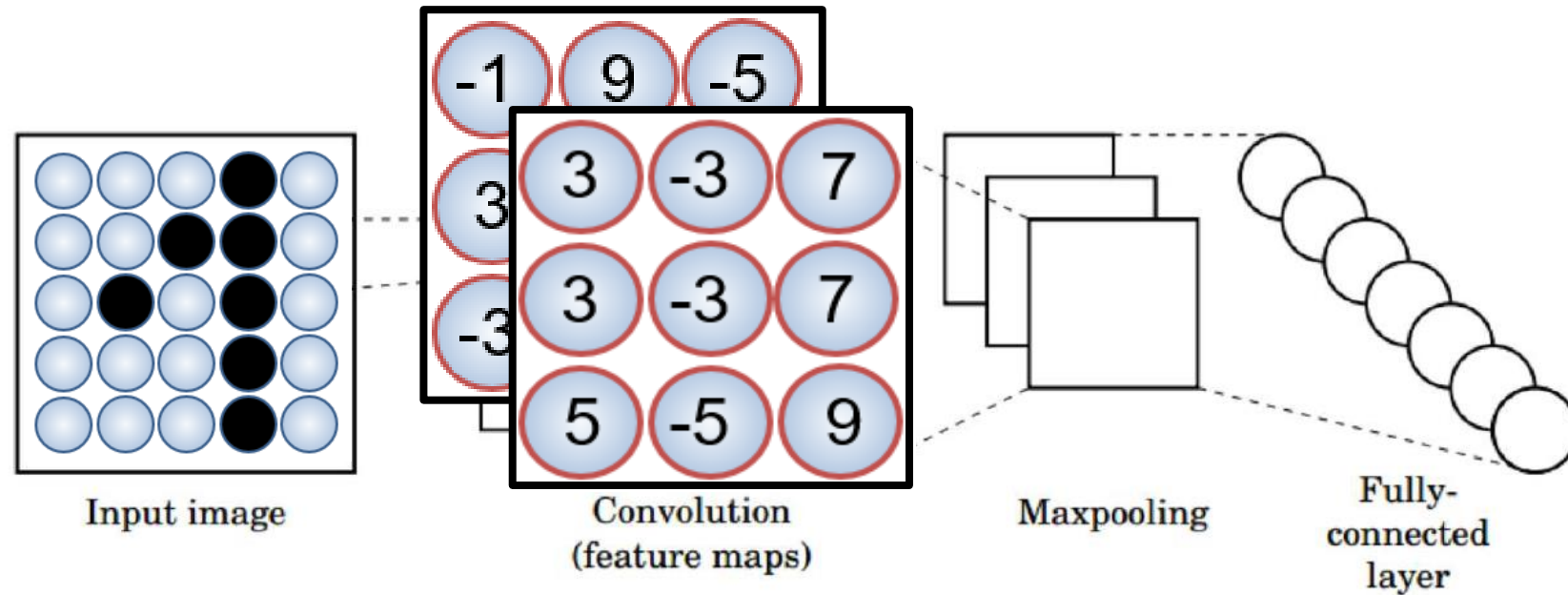
Convolutional Neural Networks

Learning – Feature Extraction: filter comparison



Convolutional Neural Networks

Building a CNN - Example

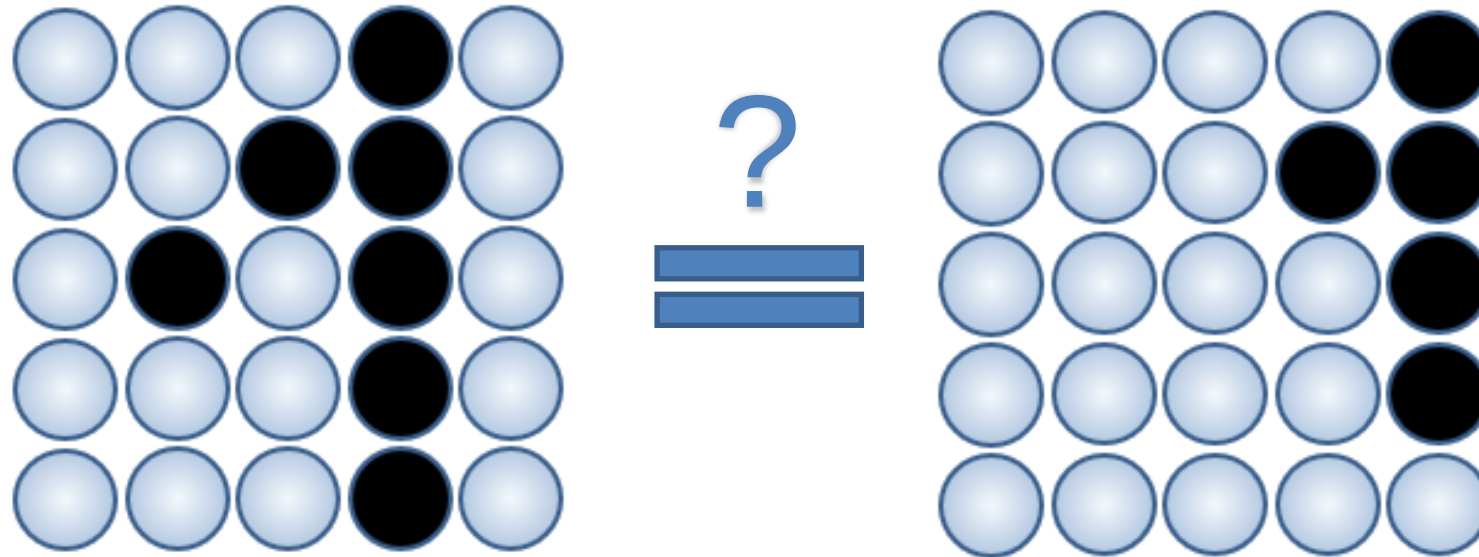


1. **Convolution:** Apply filters with learned weights to generate feature maps.
2. **Non-linearity:** Often ReLU.
3. **Pooling:** Downsampling operation on each feature map.

Train model with image data.
Learn weights of filters in convolutional layers.

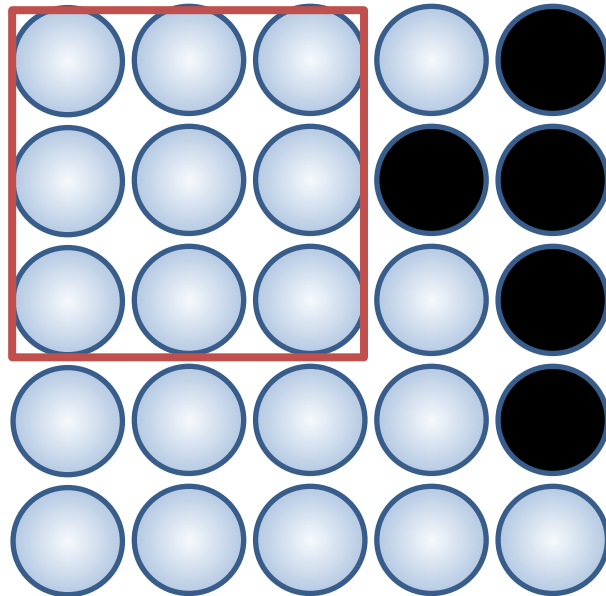
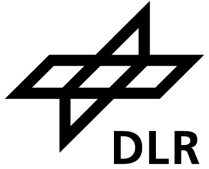
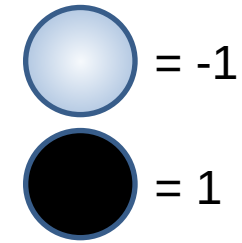
Convolutional Neural Networks

Filters to detect features



Convolutional Neural Networks

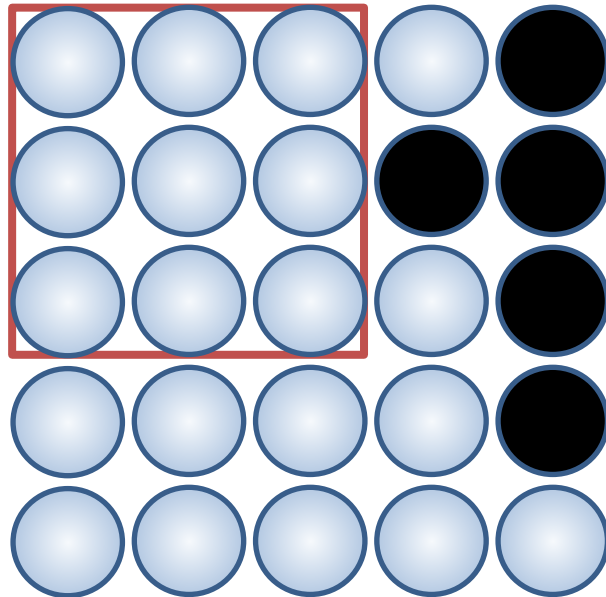
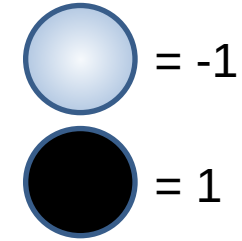
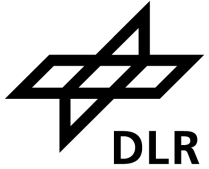
Filters to detect features



The usage of filters in detecting features allow good identification even in the event of **displacement, shrinkage, rotation or deformation**.

Convolutional Neural Networks

Filters to detect features



The usage of filters in detecting features allow good identification even in the event of **displacement, shrinkage, rotation or deformation**.

e.g. Filter $\mathbf{f}_1 =$

-1	-1	1
-1	1	1
1	-1	1

-1	-1	-1
-1	-1	-1
-1	-1	-1

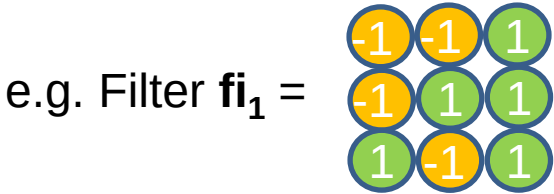
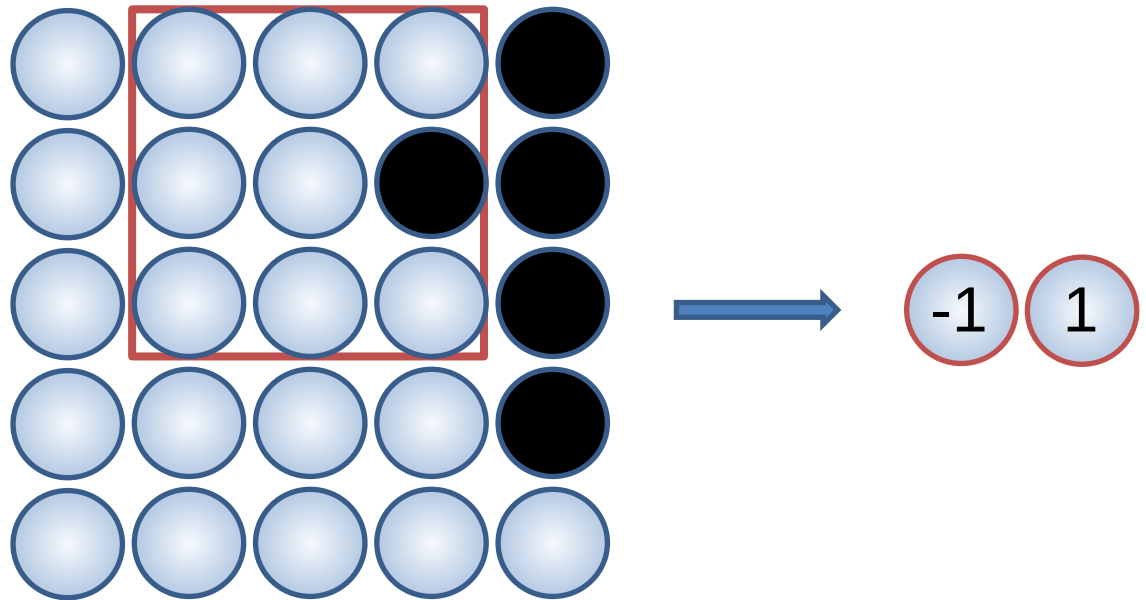
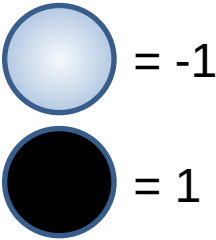
 $\times$

-1	-1	1
-1	1	1
1	-1	1

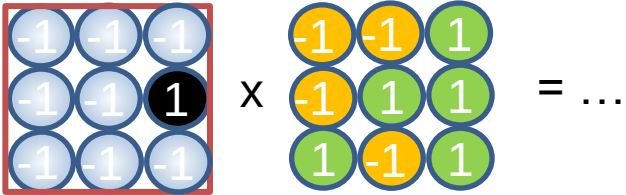
 $= \dots$

Convolutional Neural Networks

Filters to detect features

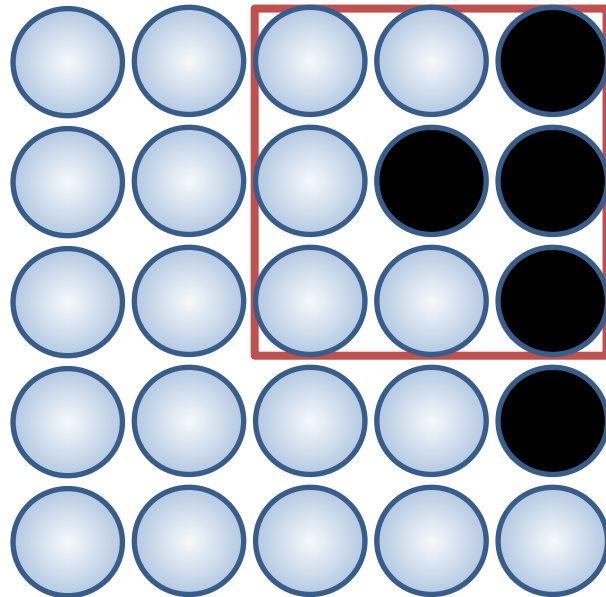
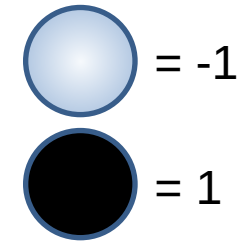
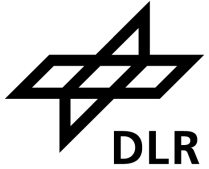


Feature/filter $\mathbf{f}_1$ still matches the deformed image well

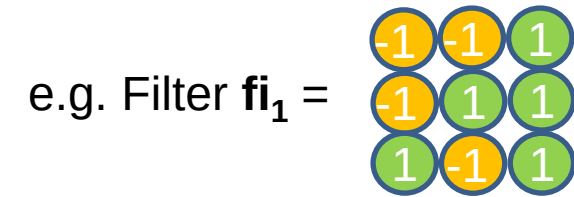
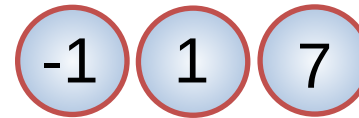


Convolutional Neural Networks

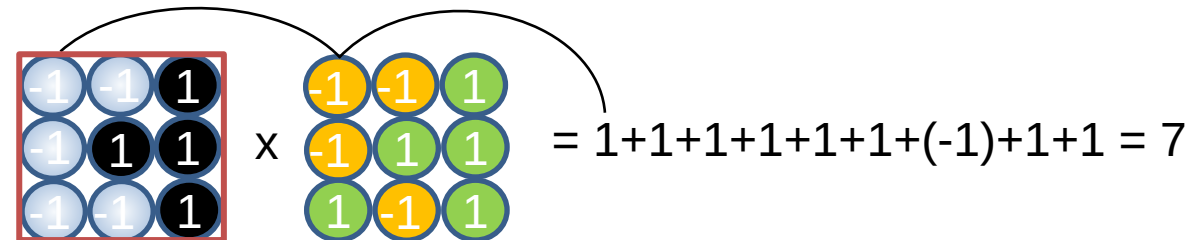
Filters to detect features



Not a full match
but still a high score

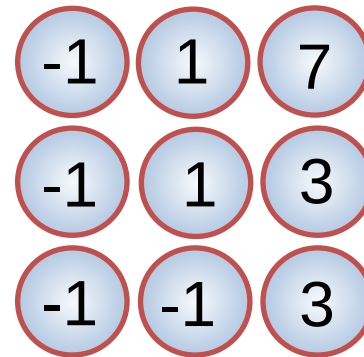
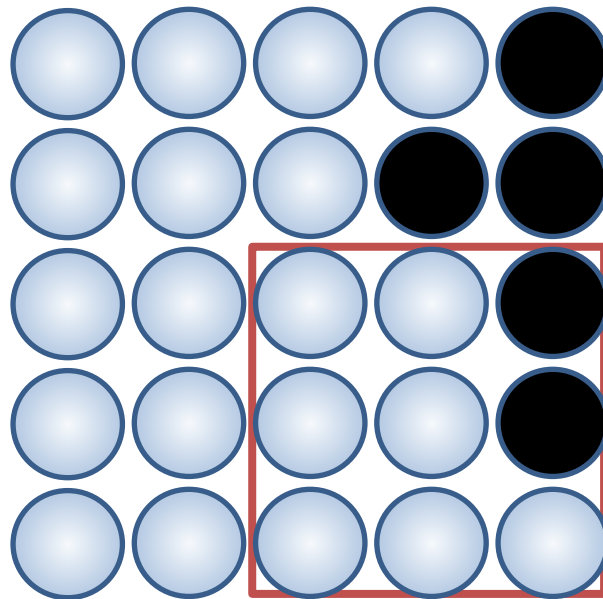
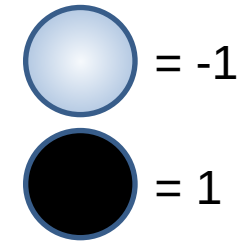


Feature/filter $\mathbf{f}_1$ still matches the deformed image well
Remember! **A full match would be 9**

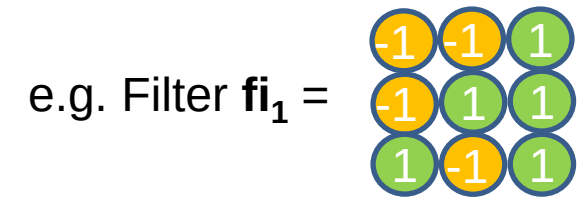


Convolutional Neural Networks

Filters to detect features

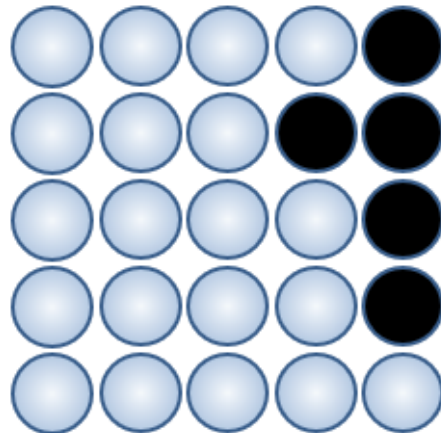
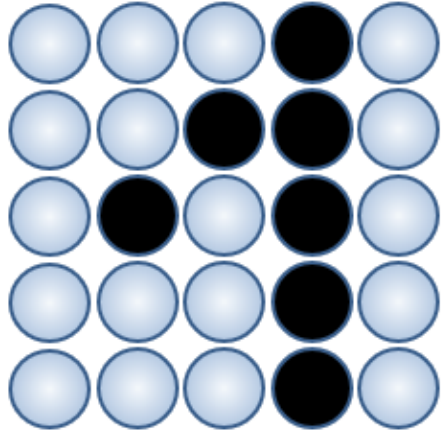


Finished feature map
using filter $\mathbf{f}_1$



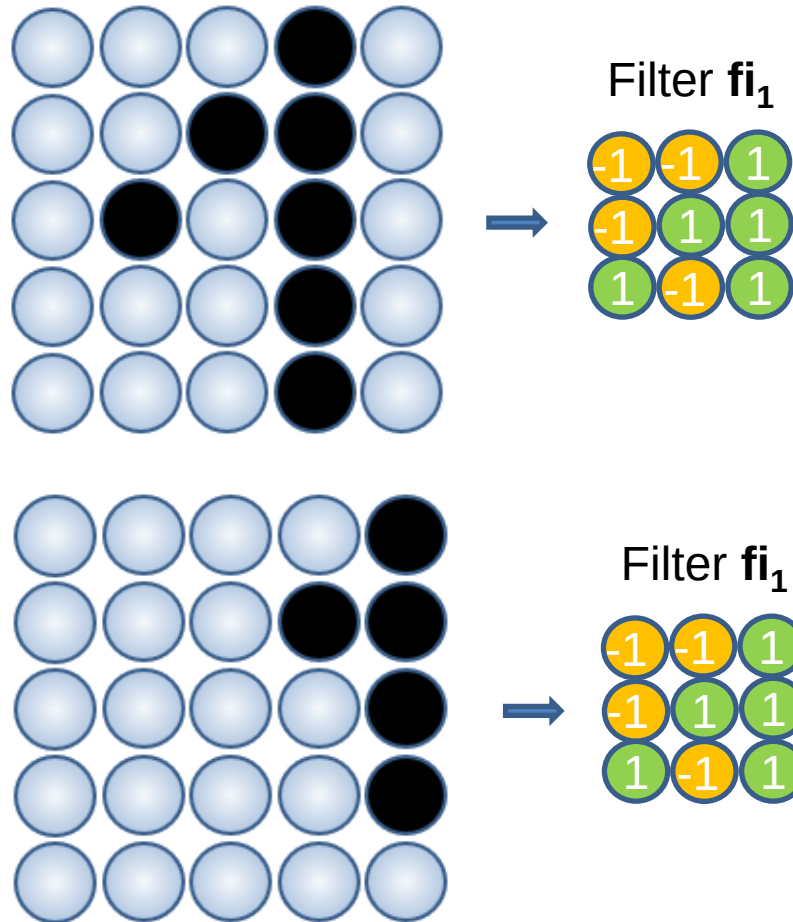
Convolutional Neural Networks

Filters to detect features: comparison



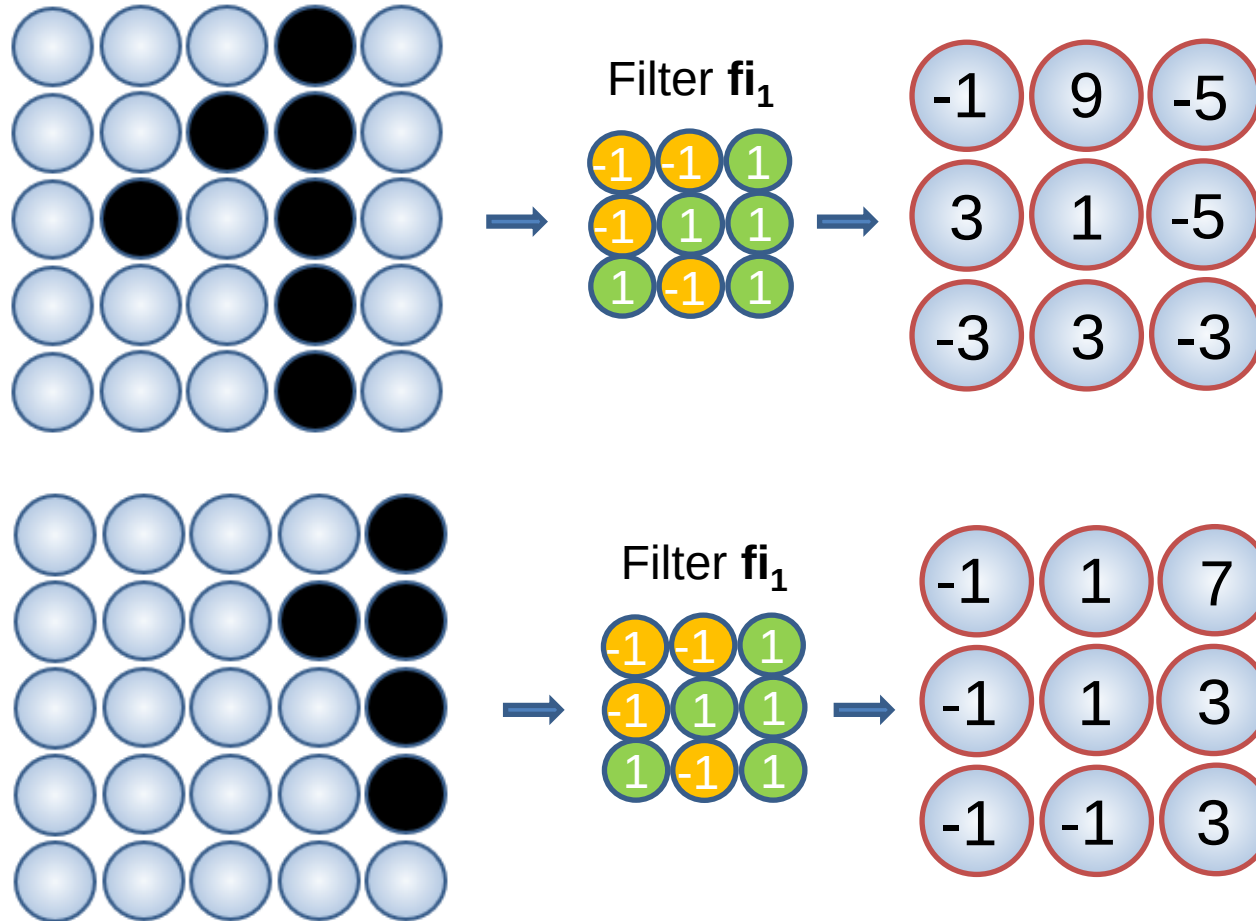
Convolutional Neural Networks

Filters to detect features: comparison



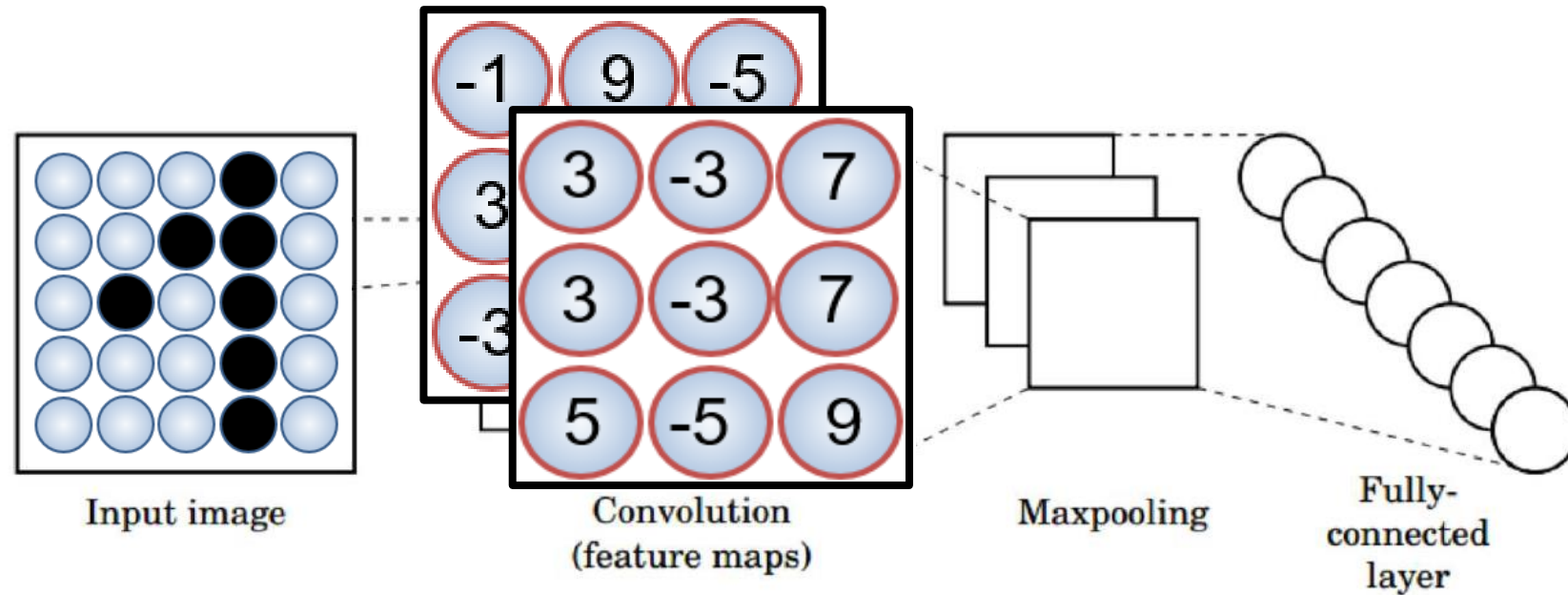
Convolutional Neural Networks

Filters to detect features: comparison



Convolutional Neural Networks

Pooling

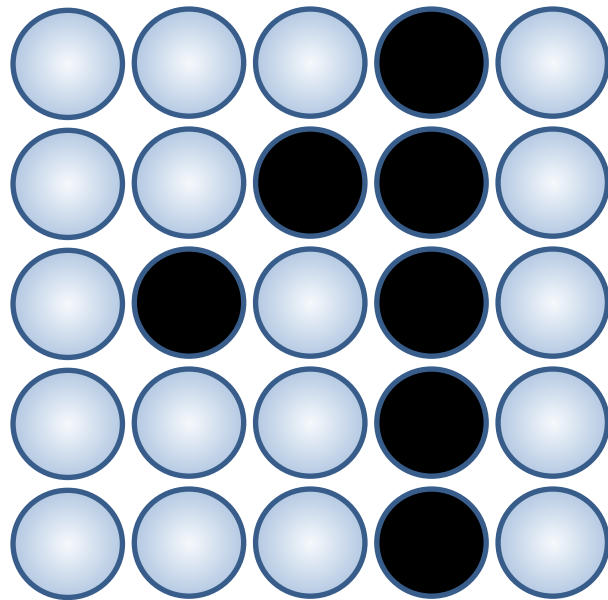


1. **Convolution:** Apply filters with learned weights to generate feature maps.
2. **Non-linearity:** Often ReLU.
3. **Pooling:** Downsampling operation on each feature map.

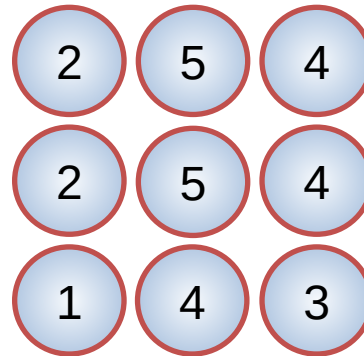
Train model with image data.
Learn weights of filters in convolutional layers.

Convolutional Neural Networks

Max Pool – Example

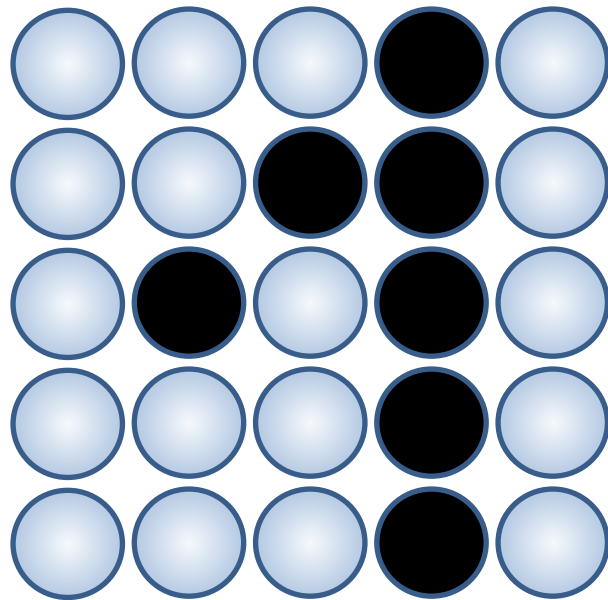


- We already have the layer with the 9 neurons for the 5x5 image

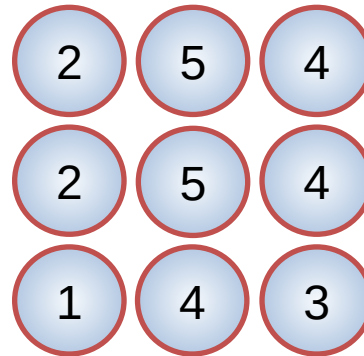


Convolutional Neural Networks

Max Pool – Example

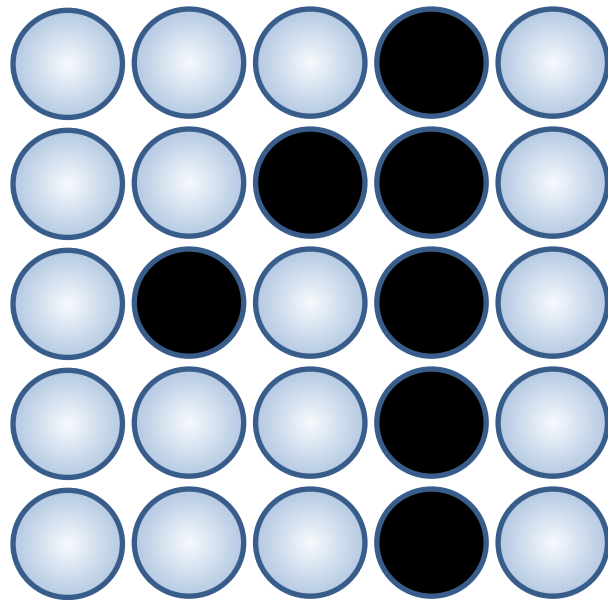


- We already have the layer with the 9 neurons for the 5x5 image
- Now we want to reduce dimensionality and spatial invariance

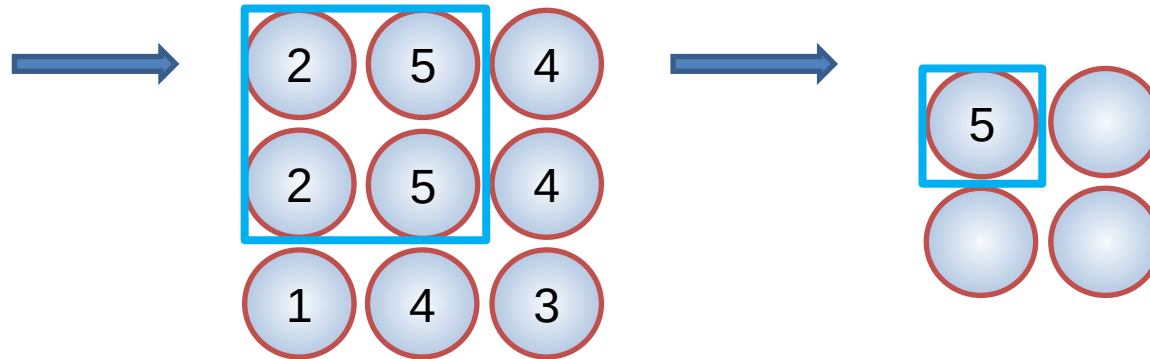


Convolutional Neural Networks

Max Pool – Example

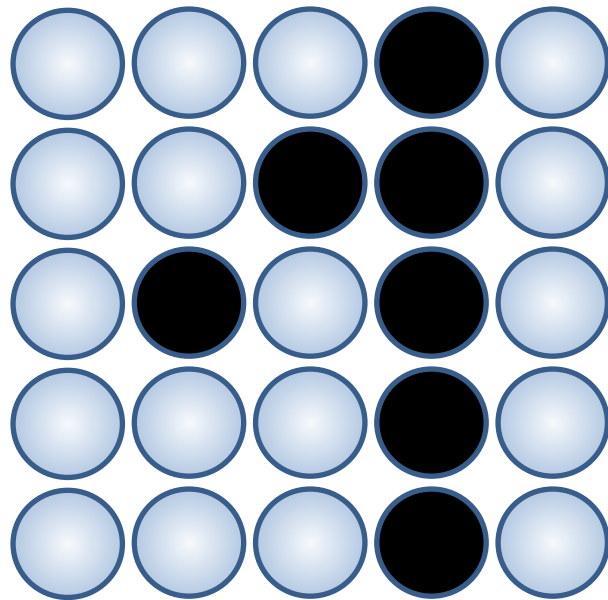


- We already have the layer with the 9 neurons for the 5x5 image
- Now we want to reduce dimensionality and spatial invariance
- Using max pool (find max with 2x2 filters and stride 1)

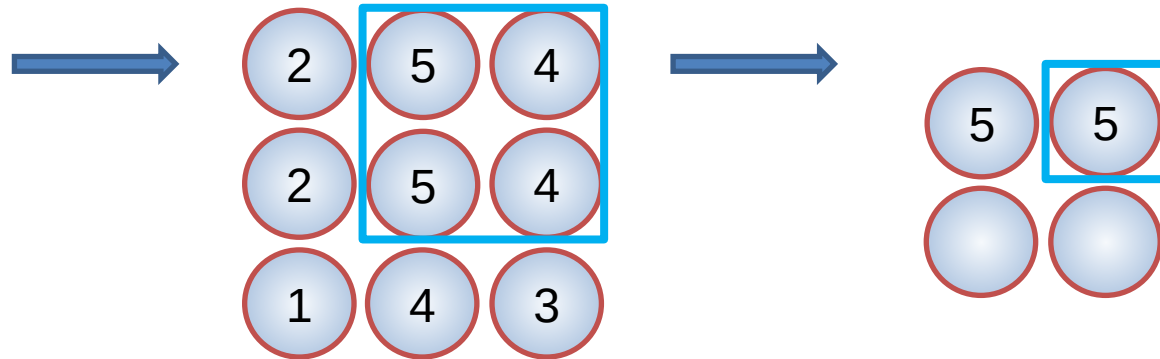


Convolutional Neural Networks

Max Pool – Example

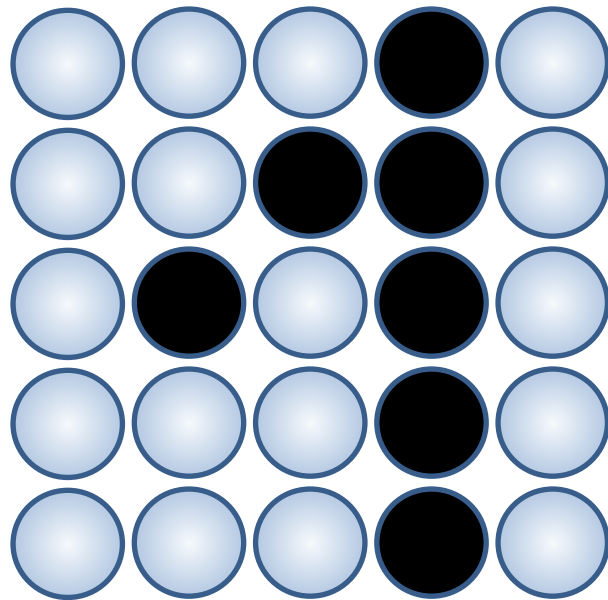


- We already have the layer with the 9 neurons for the 5x5 image
- Now we want to reduce dimensionality and spatial invariance
- Using max pool (find max with 2x2 filters and stride 1)

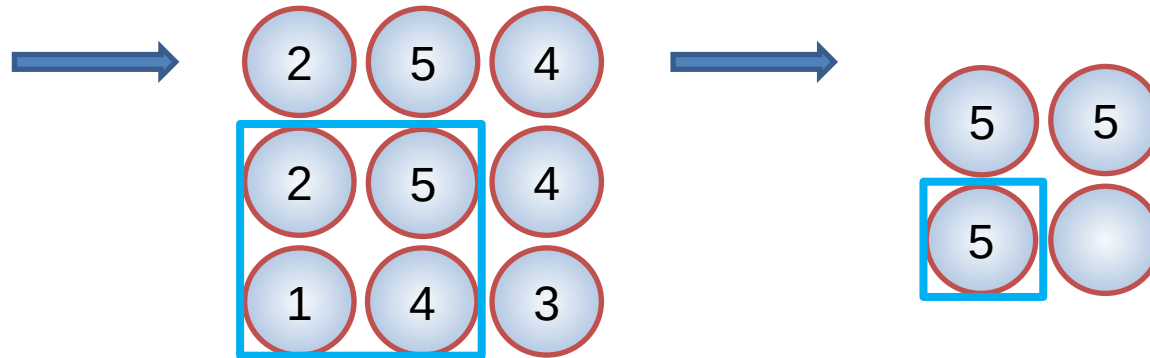


Convolutional Neural Networks

Max Pool – Example

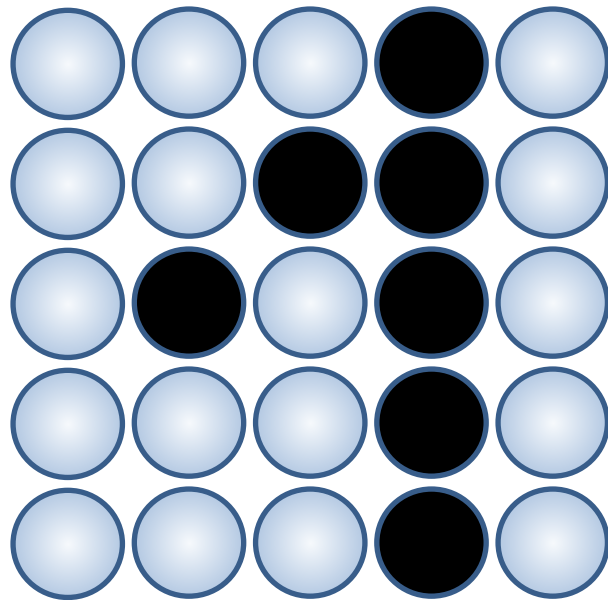


- We already have the layer with the 9 neurons for the 5x5 image
- Now we want to reduce dimensionality and spatial invariance
- Using max pool (find max with 2x2 filters and stride 1)

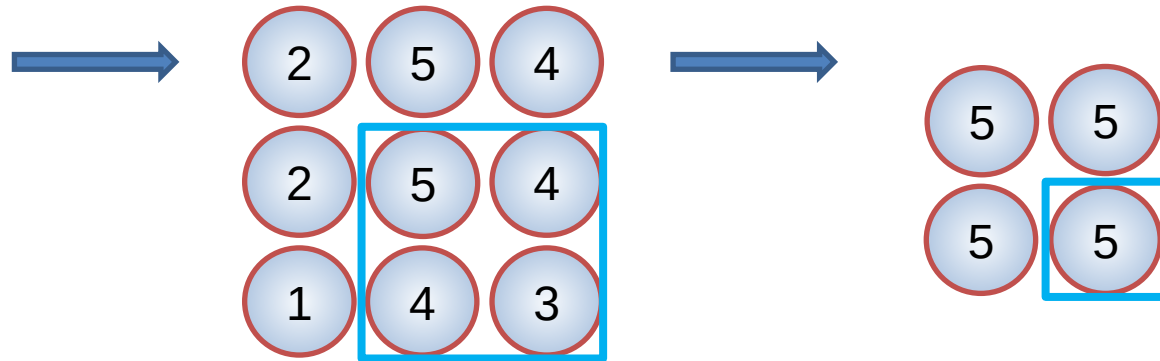


Convolutional Neural Networks

Max Pool – Example

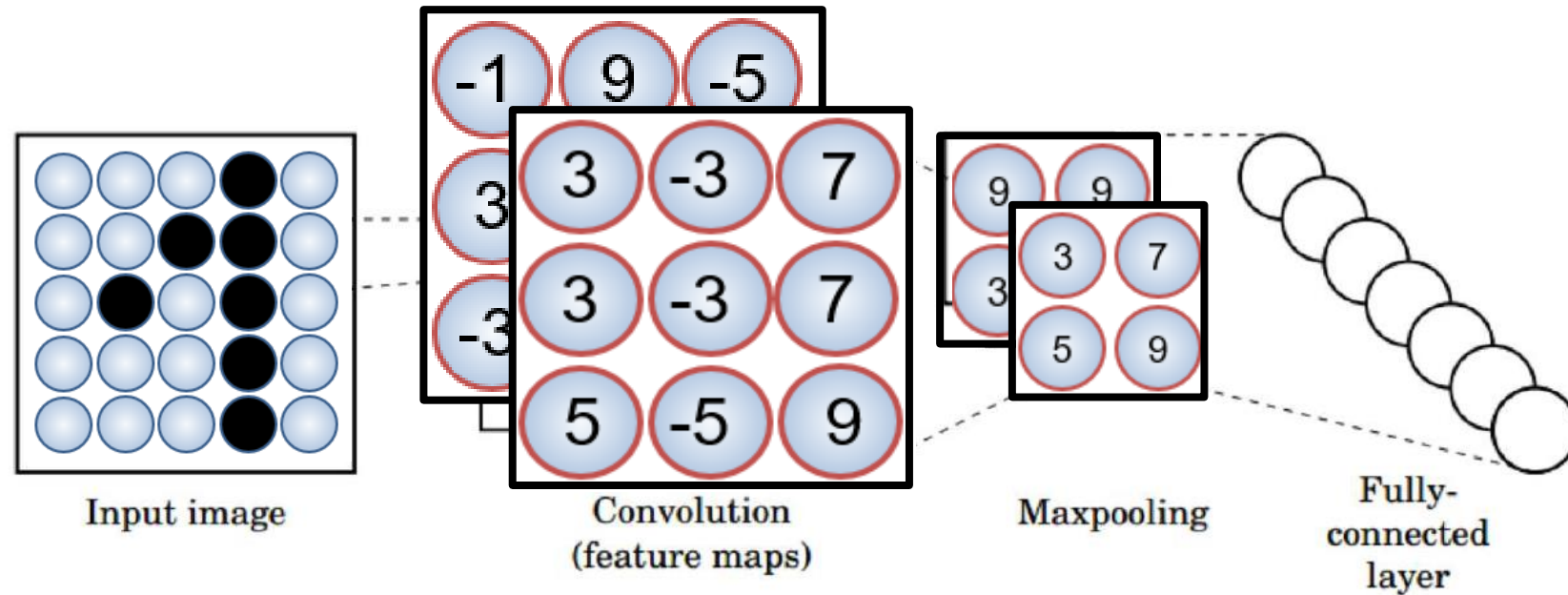


- We already have the layer with the 9 neurons for the 5x5 image
- Now we want to reduce dimensionality and spatial invariance
- Using max pool (find max with 2x2 filters and stride 1)



Convolutional Neural Networks

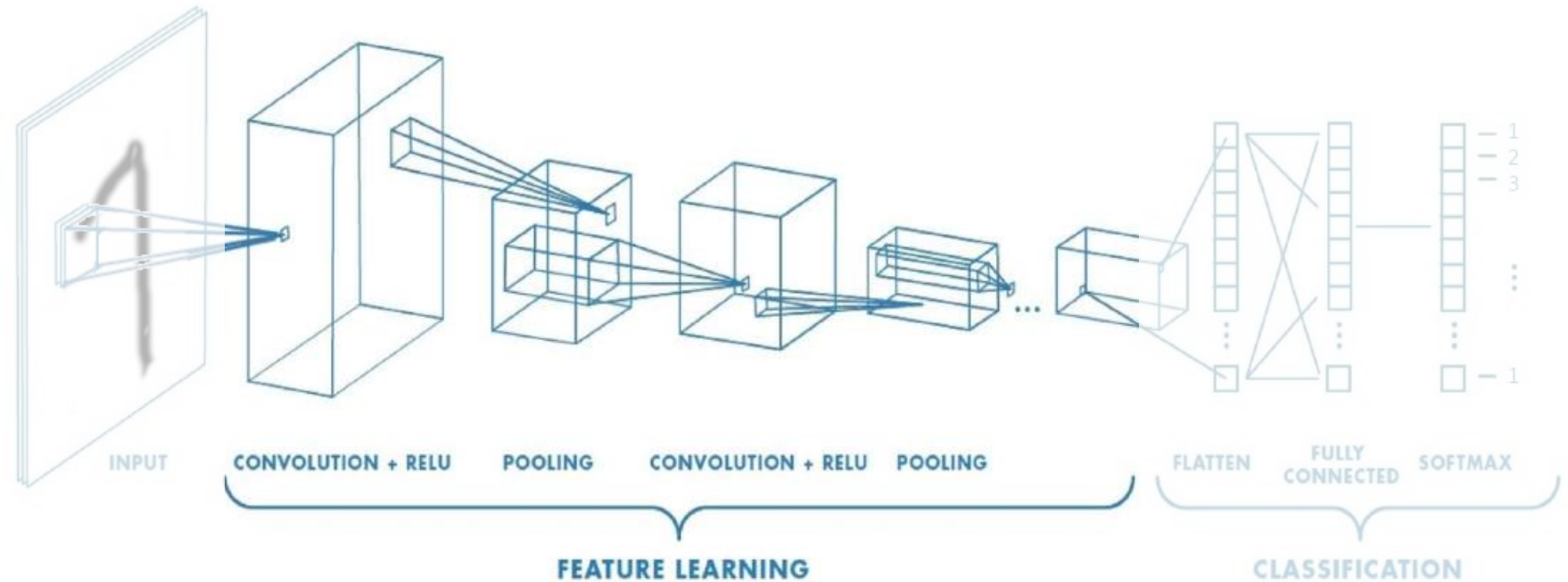
Building a CNN – Example



1. **Convolution:** Apply filters with learned weights to generate feature maps.
2. **Non-linearity:** Often ReLU.
3. **Pooling:** Downsampling operation on each feature map.

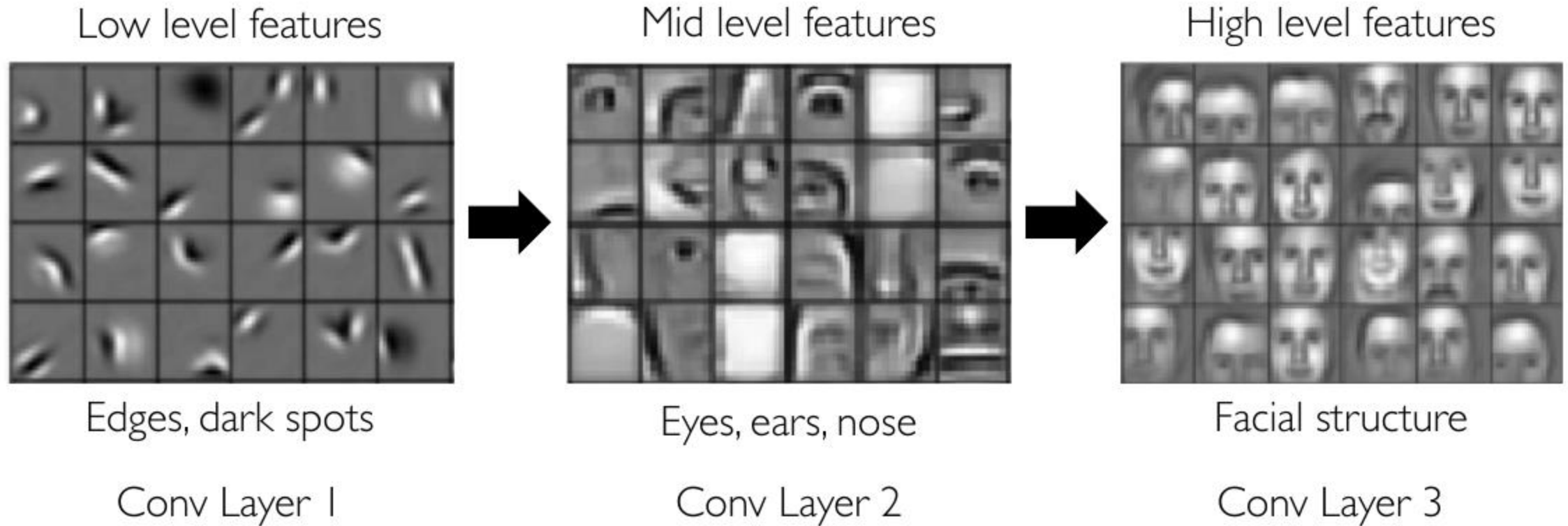
Train model with image data.
Learn weights of filters in convolutional layers.

Representation Learning

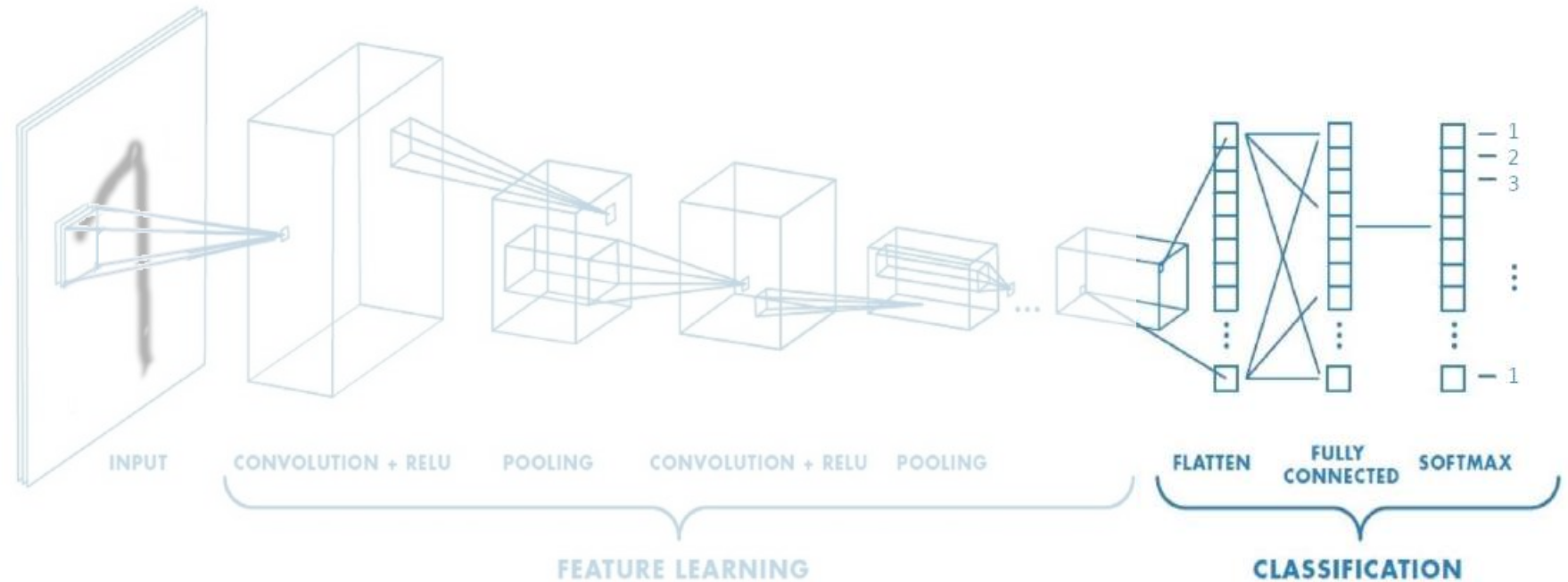


Convolutional Neural Networks

Representation Learning



Classification

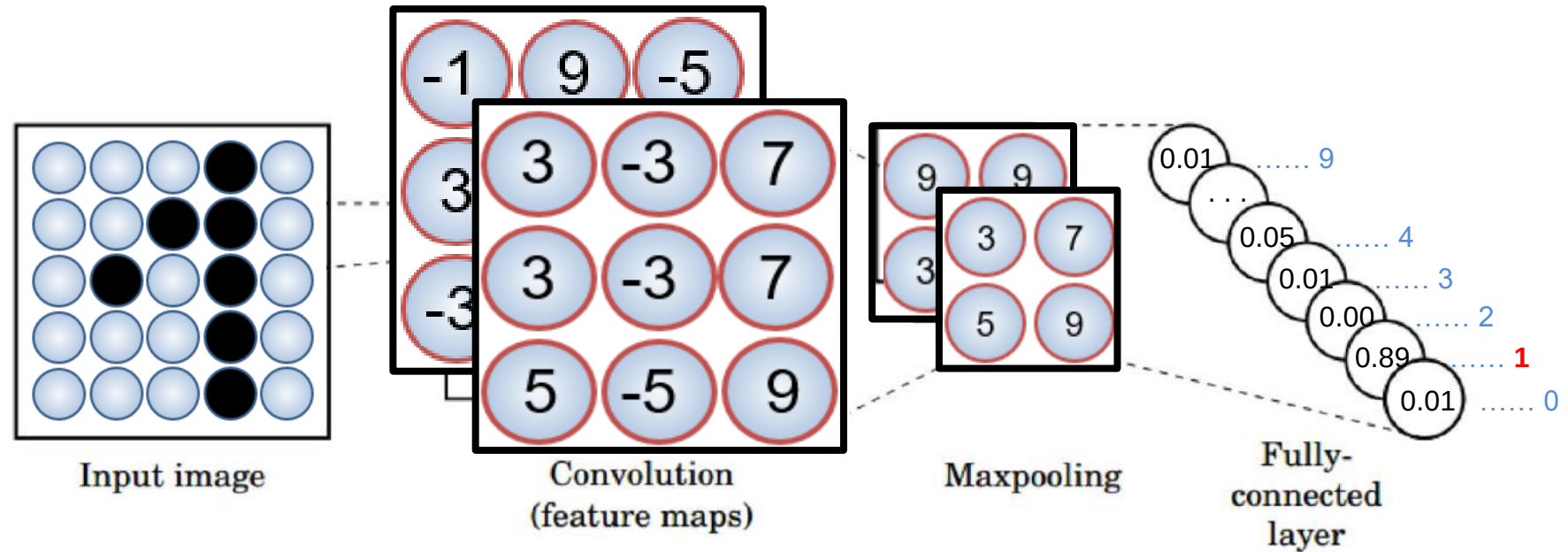


- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

Convolutional Neural Networks

Building a CNN – Example

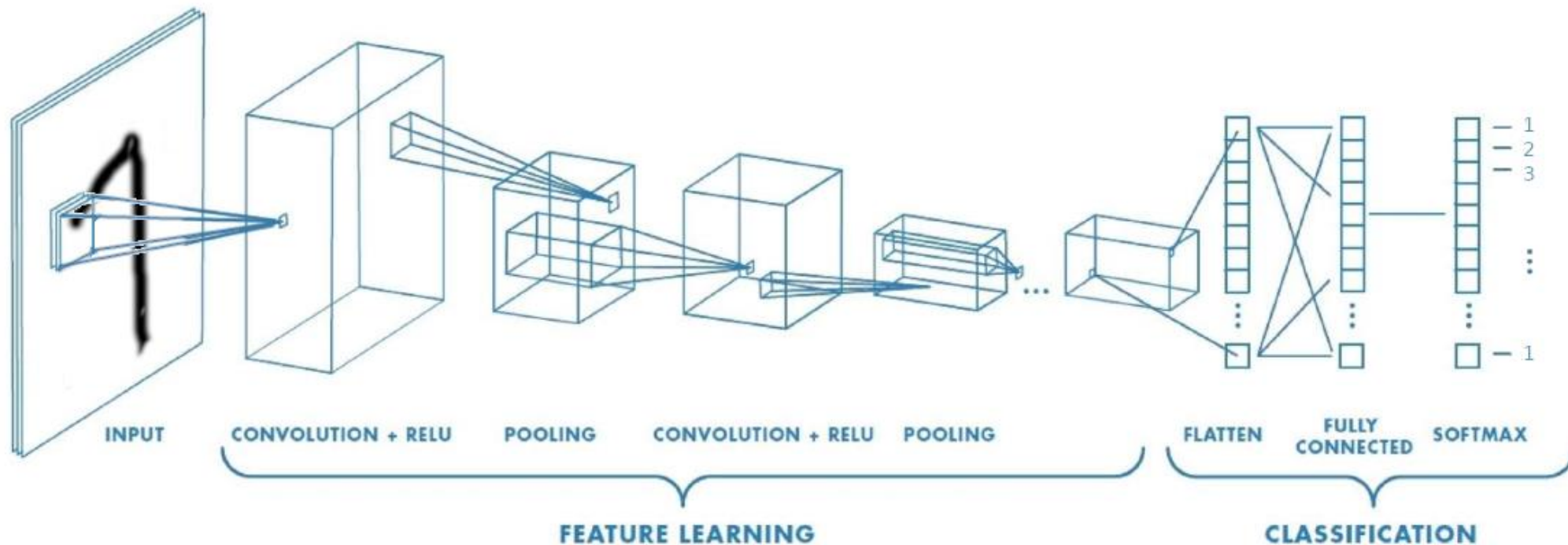


1. **Convolution:** Apply filters with learned weights to generate feature maps.
2. **Non-linearity:** Often ReLU.
3. **Pooling:** Downsampling operation on each feature map.

Train model with image data.
Learn weights of filters in convolutional layers.

Convolutional Neural Networks

Training with backpropagation

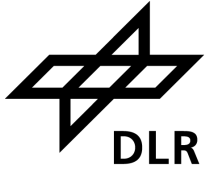


Learn weights for convolutional filters and fully connected layers

Backpropagation: cross-entropy loss

$$J(\theta) = \sum_i y^{(i)} \log(\hat{y}^{(i)})$$

Curriculum



Next:

- Hands-on

Topic: **Introduction to Deep Learning**
Part II – Advanced Concept and Convolutional Neural Network

Date: 2026-05-28

Author: Auliya Fitri, Sheetal Sahoo, Sreerag Naveenachandran

Institute: Data Science

Image sources: All images “DLR (CC BY-NC-ND 3.0)” unless otherwise stated